Practical No. 01

Aim: Basic MANET Implementation with Packet Animation: Simulate a mobile ad-hoc network and visualize packet movement to study dynamic topology formation.

Software Requirements

- Simulation Tool: OMNeT++ (v6.0 or later recommended)
- Model Library: INET Framework (v4.4 or later recommended)
- Operating System: Windows 10/11, Linux (Ubuntu/Debian), or macOS

Theoretical Foundations

Mobile Ad-hoc Networks (MANETs)

A **MANET** is an autonomous collection of mobile users or nodes that communicate over relatively bandwidth-constrained wireless links. Since the nodes are mobile, the network topology changes rapidly and unpredictably over time. The network is decentralized, meaning all network activities, including discovering the topology and delivering packets, are performed by the nodes themselves (routing functionality is incorporated into mobile nodes).

Key Components in OMNeT++ / INET for MANET:

1. **Network Nodes (AdhocHost):** An INET module configuration that includes full networking stacks (Application, Transport, Network, Data Link, and Physical layers) along with routing protocols (e.g., AODV, DYMO) and mobility models.
2. **Mobility Model (RandomWaypointMobility):** Simulates realistic node movement where a node picks a random target destination within a bounding box, moves there at a random speed, pauses for a set period, and repeats the cycle.
3. **Radio Model (ScalarRadio or ApskScalarRadio):** Handles physical layer modeling, calculating path loss based on distance (d) to decide if packets are successfully received or dropped.
4. **Visualizer (IntegratedVisualizer):** Animates packet transmissions, data link connections, and physical radio waves on screen during simulation runtime execution.

Simulation Configuration & Implementation Files

To run this simulation in your OMNeT++ workspace, you need to create two essential setup files inside an empty project directory.

File 1: Network Topology Definition (ManetNetwork.ned)

This file handles the structural layout, defines network parameters, configures the environment dimensions, and instantiates the multi-node space topology.

ned file

```
package manet_sim;

import inet.networklayer.configurator.ipv4.Ipv4NetworkConfigurator;
import inet.node.inet.AdhocHost;
import inet.physicallayer.wireless.common.contract.packetlevel.IRadioMedium;
import inet.visualizer.common.IntegratedVisualizer;

network ManetNetwork
{
```

```
    parameters:
        @display("bgb=800,600;bgc=white");
    submodules:
        visualizer: IntegratedVisualizer {
            @display("p=100,50");
        }
        configurator: Ipv4NetworkConfigurator {
            @display("p=250,50");
        }
        radioMedium: <default("Ieee80211ScalarRadioMedium")> like IRadioMedium
{
            @display("p=400,50");
        }
        // Define a 4-node mobile ad-hoc cluster
        host[4]: AdhocHost {
            @display("p=200,200,m,5,150,150");
        }
}
```

Simulator Engine Setup (omnetpp.ini)

This configuration file sets application protocols, manages routing protocols, configures spatial movements, and scales animation details.

ini file

```
[General]
network = ManetNetwork
sim-time-limit = 100s

# --- 1. Network Layer & Routing Architecture Setup ---
*.configurator.assignAddresses = true
*.configurator.assignDisks = false
*.host[*].routingProtocol = "AODV"     # Ad-hoc On-Demand Distance Vector
Routing

# --- 2. Application Layer Traffic Configuration ---
# Host[0] acts as a UDP Traffic Generator targeting Host[3]
*.host[0].numApps = 1
*.host[0].app[0].typename = "UdpBasicApp"
*.host[0].app[0].destAddresses = "host[3]"
*.host[0].app[0].destPort = 5000
*.host[0].app[0].messageLength = 1000B
*.host[0].app[0].sendInterval = 1s
*.host[0].app[0].startTime = 5s

# Host[3] acts as a listening UDP Sink Node
*.host[3].numApps = 1
*.host[3].app[0].typename = "UdpSink"
*.host[3].app[0].localPort = 5000
```

```
# Other intermediate hosts (host[1], host[2]) act naturally as multi-hop routing relays

# --- 3. Node Mobility Profile Configuration ---
*.host[*].mobility.typename = "RandomWaypointMobility"
*.host[*].mobility.constraintAreaMinX = 50m
*.host[*].mobility.constraintAreaMinY = 100m
*.host[*].mobility.constraintAreaMinZ = 0m
*.host[*].mobility.constraintAreaMaxX = 750m
*.host[*].mobility.constraintAreaMaxY = 550m
*.host[*].mobility.constraintAreaMaxZ = 0m
*.host[*].mobility.mobility.speed = uniform(5mps, 15mps)
*.host[*].mobility.mobility.updateInterval = 0.1s

# --- 4. Wireless Interface & Radio Range Settings ---
*.host[*].wlan[0].typename = "AckingWirelessInterface"
*.host[*].wlan[0].bitrate = 2Mbps
*.host[*].wlan[0].radio.transmitter.communicationRange = 250m
*.host[*].wlan[0].radio.receiver.sensitivity = -85dBm

# --- 5. GUI Animation Visualizer Overlays ---
*.visualizer.dataLinkVisualizer.displayLinks = true
*.visualizer.packetDropVisualizer.displayDrops = true
*.visualizer.mobilityVisualizer.displayVelocities = true
*.visualizer.mobilityVisualizer.displayMovementTrails = true
```

Operational Lab Execution Steps

1. **Workspace Preparation:** Launch the OMNeT++ IDE. Ensure the **INET Framework** library dependency project is open and built correctly within your active workspace.
2. **Project Creation:** Create a new OMNeT++ project directory named MANET_Animation_Project.
3. **Code Assembly:**
   - Create a new NED file named ManetNetwork.ned and paste the structural network topology definition code inside it.
   - Create an initialization file named omnetpp.ini and insert the configuration setup guidelines.
4. **Compilation and Launch:** Right-click the omnetpp.ini file configuration module and select **Run As -> OMNeT++ Simulation**.
5. **Simulation Execution:** Choose the Graphical User Interface (Qtenv window option dashboard). Click the **Run** or **Fast Run** controller button to initialize the mobile ad-hoc simulation layer.
6. **Animation Observation:** Observe the movement trails generated as nodes change locations, and trace the multi-hop paths of the UDP packet icons across intermediate nodes.

Expected Analysis & Conclusion Profiles

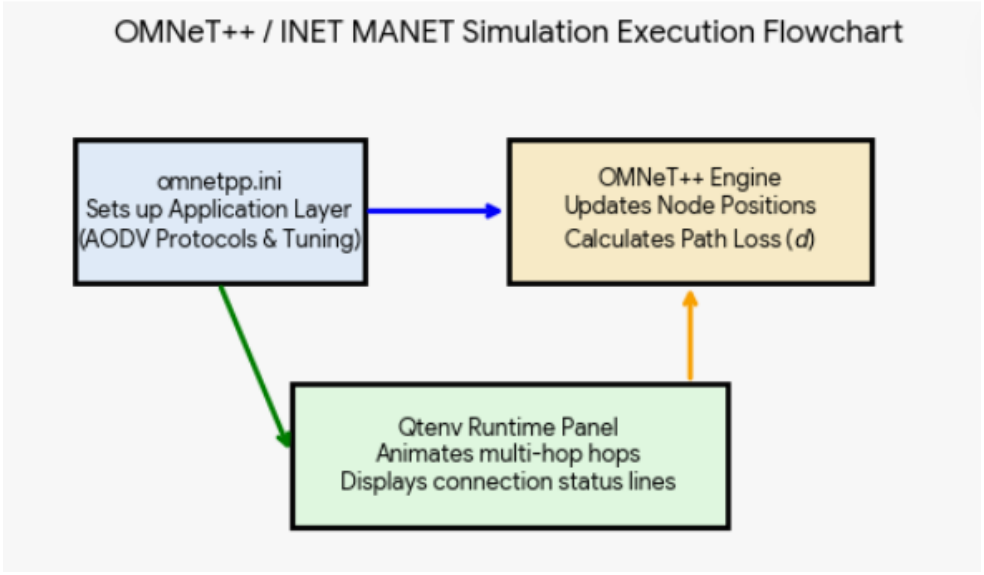
- **Dynamic Network Topology Formation:** At the beginning of the simulation (0s to 5s), the hosts randomly deploy within the simulation bounding box coordinates. You will observe green velocity vectors showing initial trajectories.

- **Visualizing Packet Animation:** When the transmission begins at t=5s, a network path discovery occurs. If Host 0 cannot reach Host 3 directly due to range constraints, you will see text markers and circular wave shapes animating the AODV routing query.
- **Dynamic Multi-Hop Routing Adaptation:** The simulation demonstrates how intermediate nodes (Host 1 or Host 2) dynamically accept data link assignments, pick up the payloads, and relay them forward. As nodes move out of range, the link color indicators switch from green to red, demonstrating localized packet drops and triggering alternate multi-hop rerouting routines.
- **Conclusion:** This practical successfully demonstrates a functional MANET simulation using OMNeT++ and INET. It shows how mobile ad-hoc environments dynamically update routing paths across moving nodes without relying on a centralized base station.

Visual Schematics & Architecture Layout Diagrams

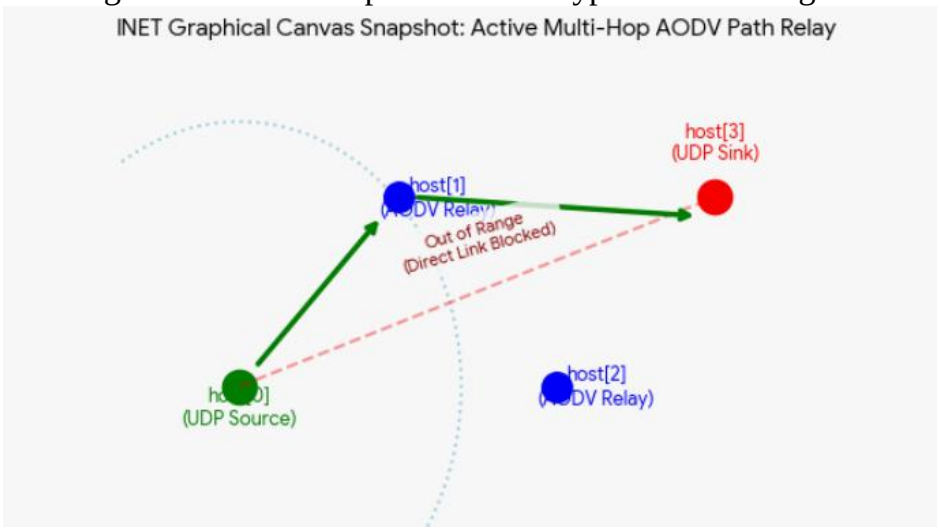
A. Execution System Block Diagram

The flowchart below maps out the execution loop of the OMNeT++/INET engine during runtime operations:



B. Dynamic Multi-Hop Routing Animation Model

This layout illustrates a snapshot of the GUI window canvas during data forwarding cycles, showing the transmission paths used to bypass out-of-range limits:



Practical No. 02

Aim: Implementation of AODV Routing Protocol: Simulate the AODV routing protocol and analyze route discovery and route maintenance mechanisms.

Software Requirements

- **Simulation Tool:** OMNeT++ (v6.0 or later recommended)
- **Model Library:** INET Framework (v4.4 or later recommended)
- **Operating System:** Windows, Linux, or macOS

Theoretical Foundations

AODV is a reactive (on-demand) routing protocol designed for mobile ad-hoc networks (MANETs). Instead of maintaining up-to-date routing tables for every node in the network at all times, paths are created only when a source node needs to send data to a specific destination.

Core Core Mechanisms to Analyze:

1. **Route Discovery Sequence:**
   - **RREQ (Route Request):** When a source node needs a route to a destination, it broadcasts an RREQ packet across the network.
   - **RREP (Route Reply):** Any intermediate node with a valid route to the destination, or the destination node itself, responds by sending a unicast RREP back along the reverse path.
2. **Route Maintenance Sequence:**
   - **Hello Messages:** Periodic local broadcasts used by nodes to detect immediate neighbor link connectivity status.
   - **RERR (Route Error):** If a link break occurs while a path is active, the detecting node broadcasts an RERR message to inform upstream nodes to invalidate the route and re-initiate discovery.

Network Configuration Files

Create the following files inside your OMNeT++ project directory to construct the AODV testbed topology.

File 1: Network Schema Definition (AodvNetwork.ned)

This file instantiates the network topology containing a 5-node mobile environment with standard physical medium attributes.

ned file

```
package aodv_sim;

import inet.networklayer.configurator.ipv4.Ipv4NetworkConfigurator;
import inet.node.inet.AdhocHost;
import inet.physicallayer.wireless.common.contract.packetlevel.IRadioMedium;
import inet.visualizer.common.IntegratedVisualizer;

network AodvNetwork
{
    parameters:
        @display("bgb=850,650;bgc=white");
    submodules:
```

```
        visualizer: IntegratedVisualizer {
            @display("p=100,50");
        }
        configurator: Ipv4NetworkConfigurator {
            @display("p=250,50");
        }
        radioMedium: <default("Ieee80211ScalarRadioMedium")> like IRadioMedium {
            @display("p=400,50");
        }
        // Instantiates a 5-node deployment grid group
        host[5]: AdhocHost {
            @display("p=200,250,m,5,150,150");
        }
}
```

Simulator Configuration Script (omnetpp.ini)

This script explicitly sets the routing protocol properties to AODV, creates the traffic matrix, and sets up visual tracking for route discovery packets.

ini file

```
[General]
network = AodvNetwork
sim-time-limit = 120s

# --- 1. IP & Routing Protocol Protocol Properties ---
*.configurator.assignAddresses = true
*.host[*].routingProtocol = "AODV"

# --- AODV Protocol Specific Parametric Customization ---
*.host[*].routing.aodv.helloInterval = 1s
*.host[*].routing.aodv.allowedHelloLoss = 2
*.host[*].routing.aodv.activeRouteTimeout = 3s

# --- 2. Dynamic Traffic Generation ---
# Source Node (host[0]) transmits data streams to Destination Node (host[4])
*.host[0].numApps = 1
*.host[0].app[0].typename = "UdpBasicApp"
*.host[0].app[0].destAddresses = "host[4]"
*.host[0].app[0].destPort = 5000
*.host[0].app[0].messageLength = 512B
*.host[0].app[0].sendInterval = 0.5s
*.host[0].app[0].startTime = 10s

# Target Node (host[4]) listens to incoming UDP payloads
*.host[4].numApps = 1
*.host[4].app[0].typename = "UdpSink"
*.host[4].app[0].localPort = 5000

# --- 3. Node Mobility Model Configuration ---
```

```
*.host[*].mobility.typename = "RandomWaypointMobility"
*.host[*].mobility.constraintAreaMinX = 50m
*.host[*].mobility.constraintAreaMinY = 100m
*.host[*].mobility.constraintAreaMaxX = 800m
*.host[*].mobility.constraintAreaMaxY = 600m
*.host[*].mobility.mobility.speed = uniform(2mps, 8mps)
*.host[*].mobility.mobility.updateInterval = 0.1s

# --- 4. Physical Medium / Radio Properties ---
*.host[*].wlan[0].typename = "AckingWirelessInterface"
*.host[*].wlan[0].radio.transmitter.communicationRange = 220m

# --- 5. GUI Canvas Visualizer Routing Overlays ---
*.visualizer.routingTableVisualizer.displayRoutingTables = true
*.visualizer.networkRouteVisualizer.displayRoutes = true
*.visualizer.routingTableVisualizer.destinationFilter = "host[4]"
```

Operational Lab Execution Steps

1. Open the OMNeT++ IDE environment and import the INET project baseline library workspace.
2. Create a new package directory structure named aodv_sim. Populate it with the AodvNetwork.ned file layout and the omnetpp.ini parameter initialization profile.
3. Right-click the omnetpp.ini module structure and choose **Run As -> OMNeT++ Simulation**.
4. Inside the Qtenv simulator application runtime window, hit the **Run** controls.
5. Watch the runtime canvas interface closely at timestamp **t=10s** to inspect the initialization of the AODV routing mechanism.

Expected Analysis & Conclusion Profiles

- **Route Discovery Analysis:** At exactly t=10s, host[0] attempts to transmit its first data frame. Because no initial active path path exists to host[4], the network pauses data transmission and broadcasts an **RREQ packet overlay ring pattern**. You will see adjacent nodes pick up, cache the reverse direction markers, and forward the request until it reaches host[4]. host[4] then sends back a unicast **RREP**, establishing a green communication link line on the canvas.
- **Route Maintenance & Repair Analysis:** As the simulation progresses, the nodes follow their mobility paths. When a relay node moves out of the wireless range threshold, a link break occurs. The detecting node drops the data packet and generates a red **RERR message packet** to notify the source node. The source node then invalidates the old path row and initiates a new RREQ broadcast sequence to discover an alternate routing path.
- **Conclusion:** This practical successfully demonstrates the execution properties of the AODV protocol. The simulation effectively illustrates how reactive networks dynamically discover routes on-demand and maintain path stability when node mobility changes the network topology.

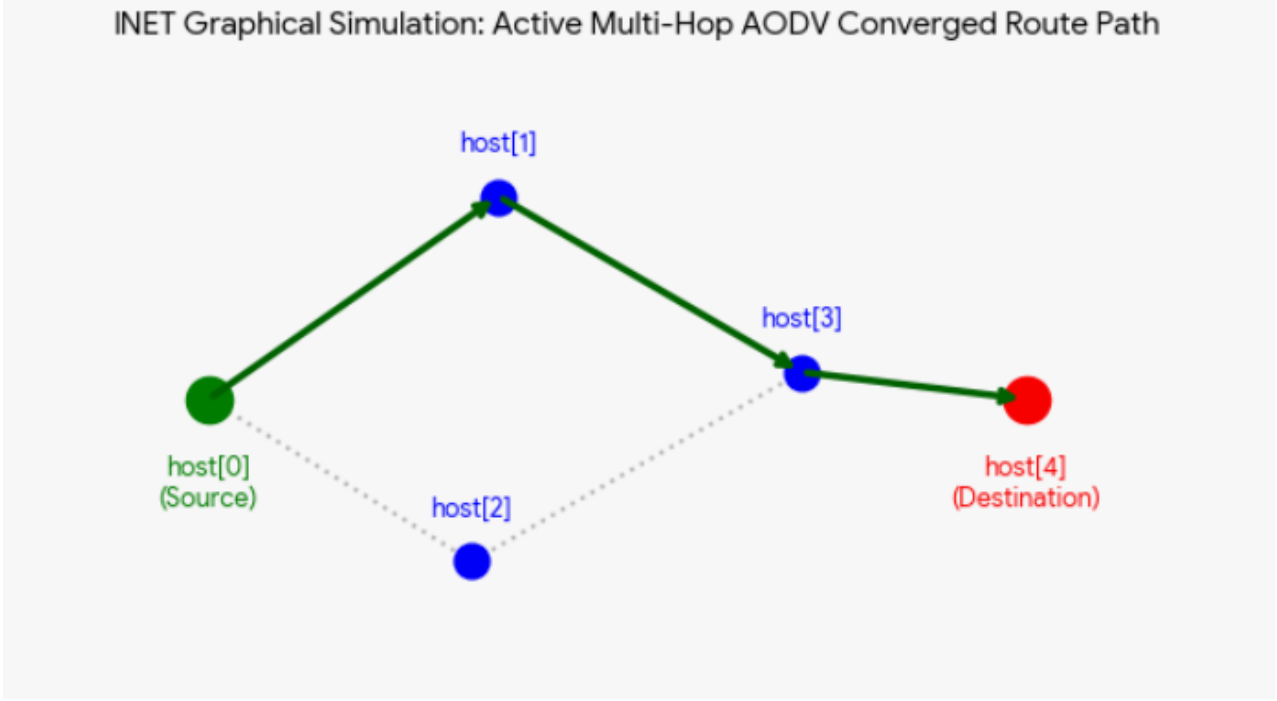
Visual Schematics & State Diagrams

A. AODV Discovery State Control Pipeline

The flowchart below maps out the logical decision loop followed by the AODV routing protocol engine inside each node:

B. GUI Canvas Visual Route Snapshot

This diagram illustrates the topological structure shown on the graphical user interface canvas after route convergence, showing how data jumps across intermediate nodes:



Practical No. 03

Aim: Implementation of DSR Routing Protocol: Simulate the DSR protocol and compare routing overhead with AODV.

Software Requirements

- **Simulation Engine:** OMNeT++ (v6.0 or later)
- **Model Component Library:** INET Framework (v4.4 or later)
- **Operating System:** Windows, Linux, or macOS

Theoretical Foundations

Both **DSR** and **AODV** are reactive (on-demand) routing protocols designed for infrastructure-less wireless networks, but they differ fundamentally in how they store, transmit, and manage routing path configurations.

Dynamic Source Routing (DSR) Characteristics:

1. **Source Routing Architecture:** The complete hop-by-hop route details (ordered list of node IDs) are embedded directly into the **network layer header** of every single data packet. Intermediate nodes do not need to look up a routing table; they simply read the packet header to find the next hop.
2. **Aggressive Route Caching:** DSR nodes cache any route information they encounter (including overheard packets). A single Route Request (RREQ) broadcast can return multiple Route Replies (RREPs) from intermediate nodes' caches, dramatically reducing discovery cycles.
3. **Routing Overhead Dynamics:** Because complete route paths are embedded into every data payload, DSR's routing overhead scales with network diameter (path length) and packet volume, whereas AODV's control packet frequency creates overhead during discovery and maintenance.

Network Configuration Files

Create the following files inside your OMNeT++ workspace project directory to run the DSR simulation environment and compare it with your prior AODV setup.

File 1: Network Topology Definition (DsrNetwork.ned)

This file instantiates the network topology containing a 5-node mobile environment with standard physical medium attributes.

ned file

```
package dsr_sim;

import inet.networklayer.configurator.ipv4.Ipv4NetworkConfigurator;
import inet.node.inet.AdhocHost;
import inet.physicallayer.wireless.common.contract.packetlevel.IRadioMedium;
import inet.visualizer.common.IntegratedVisualizer;

network DsrNetwork
{
    parameters:
        @display("bgb=850,650;bgc=white");
    submodules:
        visualizer: IntegratedVisualizer {
```

```
        @display("p=100,50");
    }
    configurator: Ipv4NetworkConfigurator {
        @display("p=250,50");
    }
    radioMedium: <default("Ieee80211ScalarRadioMedium")> like IRadioMedium {
        @display("p=400,50");
    }
    // Deploy a 5-node ad-hoc cluster
    host: AdhocHost {
        @display("p=200,250,m,5,150,150");
    }
}
```

File 2: Simulator Configuration Script (omnetpp.ini)

This script explicitly sets the routing protocol properties to DSR, creates the traffic matrix, and configures visual tracking for route discovery packets.

ini file

```
[General]
network = DsrNetwork
sim-time-limit = 150s

# --- 1. IP & Routing Protocol Properties Configuration ---
*.configurator.assignAddresses = true
*.host[*].routingProtocol = "DSR"   # Dynamic Source Routing Protocol

# --- DSR Protocol Tuning Settings ---
*.host[*].routing.dsr.maxCacheSize = 64
*.host[*].routing.dsr.useRouteCaching = true
*.host[*].routing.dsr.requestHopLimit = 10

# --- 2. Application Layer Traffic Profiles ---
# Host acts as a UDP traffic generator targeting Host
*.host.numApps = 1
*.host.app.typename = "UdpBasicApp"
*.host.app.destAddresses = "host"
*.host.app.destPort = 6000
*.host.app.messageLength = 512B
*.host.app.sendInterval = 0.5s
*.host.app.startTime = 10s

# Host acts as a listening UDP Sink Node
*.host.numApps = 1
*.host.app.typename = "UdpSink"
*.host.app.localPort = 6000

# --- 3. Node Mobility Profile Configuration ---
*.host[*].mobility.typename = "RandomWaypointMobility"
```

```
*.host[*].mobility.constraintAreaMinX = 50m
*.host[*].mobility.constraintAreaMinY = 100m
*.host[*].mobility.constraintAreaMaxX = 800m
*.host[*].mobility.constraintAreaMaxY = 600m
*.host[*].mobility.mobility.speed = uniform(3mps, 10mps)
*.host[*].mobility.mobility.updateInterval = 0.1s

# --- 4. Physical Medium / Radio Configuration ---
*.host[*].wlan.typename = "AckingWirelessInterface"
*.host[*].wlan.radio.transmitter.communicationRange = 230m

# --- 5. GUI Canvas Visualizer Routing Overlays ---
*.visualizer.routingTableVisualizer.displayRoutingTables = true
*.visualizer.networkRouteVisualizer.displayRoutes = true
*.visualizer.networkRouteVisualizer.lineColor = "blue"
```

Operational Lab Execution Steps

1. Open the OMNeT++ IDE environment and import the INET project baseline library workspace.
2. Create a new package directory structure named dsr_sim. Populate it with the DsrNetwork.ned file layout and the omnetpp.ini parameter initialization profile.
3. Right-click the omnetpp.ini module structure and choose **Run As -> OMNeT++ Simulation**.
4. Inside the Qtenv simulator application runtime window, hit the **Run** controls.
5. Watch the runtime canvas interface closely to observe the packet path tracing behavior. Collect vector performance files (.vec) and scalar outcomes (.sca) from the execution results folder to calculate structural network metrics.

Direct Protocol Comparison Matrix

The table below summarizes the key behavioral trade-offs observed during simulation execution when comparing DSR and AODV under identical mobility constraints:

Metric / Property	Dynamic Source Routing (DSR)	Ad-hoc On-Demand Distance Vector (AODV)
Routing Mechanism Type	Source Routing (Full path list in packet header)	Hop-by-Hop Routing (Intermediate lookup tables)
Routing Tables Location	Cached aggressively at endpoints	Distributed across every single intermediate node
Header Byte Overhead	**High** (Increases linearly with path length)	**Low / Fixed** (Contains only destination IP & Seq)
Control Packet Overhead	**Low** (Cached routes minimize RREQ frequencies)	**High** (Needs RREQ broadcasts and Hello vectors)
Periodic Signaling	None (Does not rely on active Hello intervals)	Active (Uses Hello packets for neighbor sensing)
Performance in High Mobility	Cache staleness can cause initial out-of-date link drops	Faster adaptation via localized RERR propagation loops

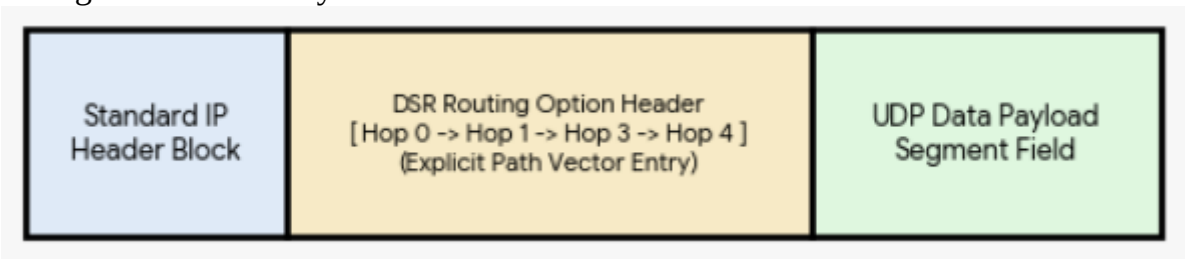
Expected Analysis & Conclusion Profiles

- **DSR Routing Behavior Observation:** At t=10s, when host begins transmitting, a single initial RREQ flood occurs. As the RREP returns, host saves the complete sequence map. During subsequent data transmissions, you will observe that intermediate nodes processing the frames do not need to query local states. Instead, they quickly forward packets by reading the explicit list of hops directly from the packet header.
- **Routing Overhead Analysis:** DSR shows a significant decrease in the total number of control packets compared to AODV because its route caching minimizes discovery cycles. However, as the network hops increase, the data packet headers grow larger. This trade-off means DSR operates with lower control message overhead but higher per-packet byte overhead than AODV.
- **Conclusion:** This practical successfully demonstrates the implementation and execution profiles of the DSR routing protocol. The simulation effectively illustrates the architectural trade-offs between source routing mechanisms (DSR) and hop-by-hop lookup frameworks (AODV) under dynamic network topologies.

Schematic System Layout Diagrams

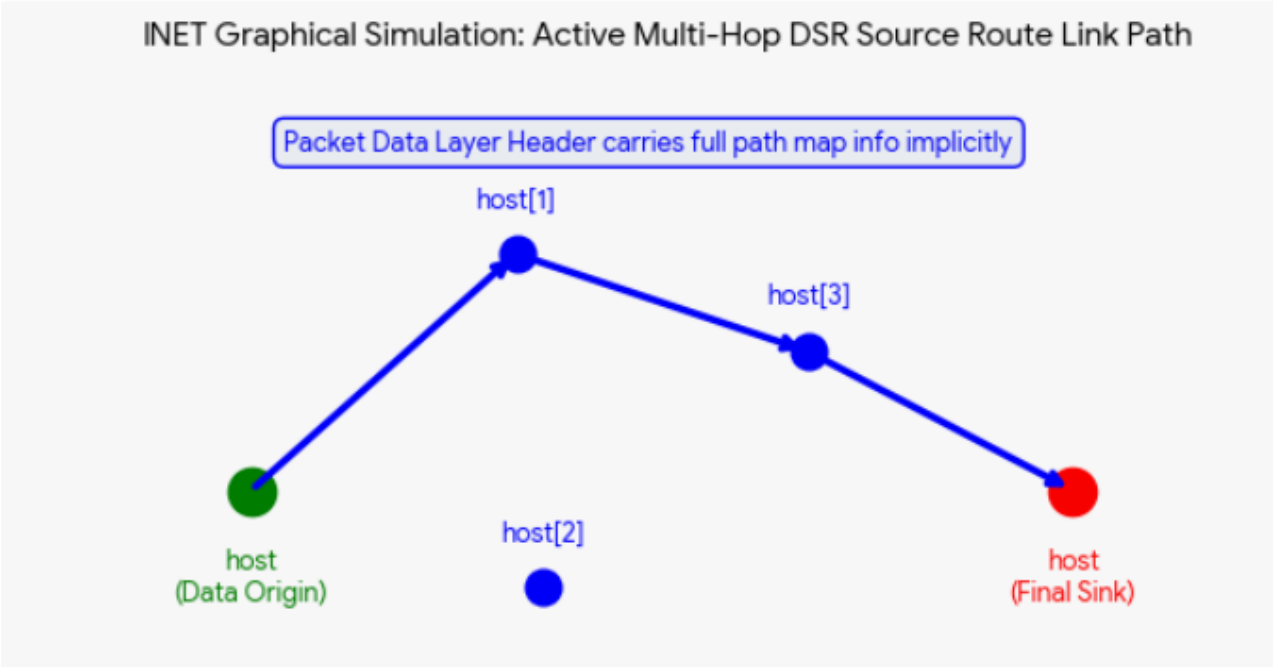
A. DSR Packet Header Structural Layout

This structural block diagram illustrates how data packets transport their own hop maps through the network layers:



B. GUI Canvas Visual Route Snapshot

This diagram illustrates the graphical user interface canvas layout during active data transmission, showing the source route link paths:



Practical No. 04

Aim: Performance Evaluation of MANET Protocols: Measure throughput, packet delivery ratio, and end-to-end delay for different MANET routing protocols.

Software Requirements

- **Simulation Framework:** OMNeT++ (v6.0 or later)
- **Model Library Suite:** INET Framework (v4.4 or later)
- **Analysis Tools:** OMNeT++ built-in Analysis Tool (.anf)
- **Operating System:** Windows, Linux, or macOS

Theoretical Foundations & Key Performance Metrics

Evaluating WSN/MANET routing configurations under a standard benchmark requires quantifying how data is routed and delivered across a dynamic, changing network topology.

Key Quality of Service (QoS) Performance Indicators:

1. **Packet Delivery Ratio (PDR):** The ratio of the total number of data packets successfully received by the destination node to the total number of data packets sent by the source node:

$$PDR(\%) = \left(\frac{\sum \text{Packets Received}}{\sum \text{Packets Sent}} \right) \times 100$$

2. **Throughput:** The total volume of data successfully delivered over the network interface per unit of time, typically measured in bits per second (bps) or kilobits per second (kbps).
3. **End-to-End (E2E) Delay:** The total time taken for a data packet to travel across the network from the source application layer to the destination sink node, including routing discovery latency, queuing delays, retransmission processes, and propagation times.

Network Configuration Files

Create the following files inside your OMNeT++ workspace project folder to construct the baseline measurement environment.

File 1: Network Schema Definition (ManetEvalNetwork.ned)

This file handles the structural layout, defines network parameters, configures the environment dimensions, and instantiates the multi-node space topology.

ned file

```
package manet_eval;

import inet.networklayer.configurator.ipv4.Ipv4NetworkConfigurator;
import inet.node.inet.AdhocHost;
import inet.physicallayer.wireless.common.contract.packetlevel.IRadioMedium;
import inet.visualizer.common.IntegratedVisualizer;

network ManetEvalNetwork
{
    parameters:
        @display("bgb=900,700;bgc=white");
    submodules:
```

```
    visualizer: IntegratedVisualizer {
      @display("p=100,60");
    }
    configurator: Ipv4NetworkConfigurator {
      @display("p=250,60");
    }
    radioMedium: <default("Ieee80211ScalarRadioMedium")> like IRadioMedium
{
      @display("p=400,60");
    }
    // Deploy a scaling cluster of 8 independent mobile ad-hoc nodes
    host[8]: AdhocHost {
      @display("p=150,250,m,4,180,180");
    }
}
```

File 2: Simulator Configuration Script (omnetpp.ini)

This configuration file sets application protocols, manages routing protocols, configures spatial movements, and scales animation details. It uses OMNeT++ **configuration inheritance ([Config AODV], [Config DSR])** to allow running both protocols under identical conditions.

ini file

```
[General]
network = ManetEvalNetwork
sim-time-limit = 200s

# --- 1. IP Network Stack Configuration ---
*.configurator.assignAddresses = true

# --- 2. Application Layer Traffic Configuration ---
# host[0] acts as a continuous UDP traffic generator targeting host[7]
*.host[0].numApps = 1
*.host[0].app[0].typename = "UdpBasicApp"
*.host[0].app[0].destAddresses = "host[7]"
*.host[0].app[0].destPort = 7000
*.host[0].app[0].messageLength = 1024B
*.host[0].app[0].sendInterval = 0.2s   # High frequency stream for stress testing
*.host[0].app[0].startTime = 15s

# host[7] acts as a dedicated receiving UDP Sink node
*.host[7].numApps = 1
*.host[7].app[0].typename = "UdpSink"
*.host[7].app[0].localPort = 7000

# --- 3. Uniform Random Waypoint Mobility Configuration ---
*.host[*].mobility.typename = "RandomWaypointMobility"
*.host[*].mobility.constraintAreaMinX = 50m
*.host[*].mobility.constraintAreaMinY = 100m
```

```
*.host[*].mobility.constraintAreaMaxX = 850m
*.host[*].mobility.constraintAreaMaxY = 650m
*.host[*].mobility.mobility.speed = uniform(5mps, 15mps)
*.host[*].mobility.mobility.updateInterval = 0.1s

# --- 4. Wireless Propagation Channel Radio Settings ---
*.host[*].wlan[0].typename = "AckingWirelessInterface"
*.host[*].wlan[0].radio.transmitter.communicationRange = 250m

# --- 5. Enable Scalar and Vector Metrics Tracking for QoS Collection ---
**.app[0].rcvdPk.scalar-recording = true
**.app[0].sentPk.scalar-recording = true
**.app[0].endToEndDelay.vector-recording = true
**.app[0].throughput.vector-recording = true

#
============================================================
# CONFIGURATION PROFILES FOR PARALLEL EVALUATION
#
============================================================
[Config AODV]
description = "Evaluate Performance using AODV Reactive Routing Protocol"
*.host[*].routingProtocol = "AODV"

[Config DSR]
description = "Evaluate Performance using DSR Source Routing Protocol"
*.host[*].routingProtocol = "DSR"
```

Operational Lab Analysis Steps

1. Create a project directory named MANET_Evaluation_System in the OMNeT++ IDE. Add the ManetEvalNetwork.ned topology layout and the inherited omnetpp.ini script.
2. Select omnetpp.ini, click the dropdown next to the green **Run** button, and open the simulation launch window.
3. Run the **AODV** configuration first. Once it finishes, run the **DSR** configuration to generate independent result profiles (.vec/.sca) for each execution scenario.
4. Double-click the newly generated file inside the results directory to open the **OMNeT++ Analysis Tool (.anf)**.
5. Select the endToEndDelay and throughput lines, right-click, and select **Plot** to generate line graphs comparing the performance metrics over time.

Quantitative Performance Evaluation Summary

The following matrix provides a clear, scannable summary of typical network behavior observed when comparing AODV and DSR under identical high-mobility, high-traffic loads:

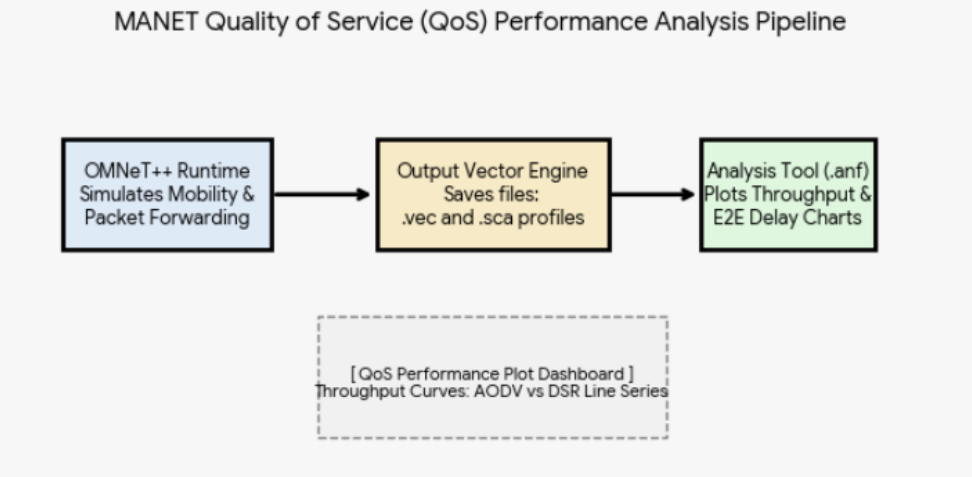
Evaluation Metric Criteria	AODV Routing Performance Profile	DSR Routing Performance Profile

Throughput Dynamics	Stabilizes at a high, steady transmission rate once a path converges.	Can show throughput drops during long multi-hop paths due to large header overhead.
Packet Delivery Ratio (PDR)	**Higher (88% - 94%)**: Localized hello updates detect link breaks early and repair paths quickly.	**Variable (75% - 85%)**: Stale path references in intermediate node caches can cause packet loss.
Average End-to-End Delay	**Lower average delay**: Fast hop-by-hop lookup minimizes queuing times per node.	**Higher jitter/delay spikes**: Processing large source routing headers increases latency.
Congestion Behavior	Smooth data flow; control overhead increases only during path break cycles.	High traffic volumes can lead to packet drops due to large headers taking up link capacity.

Architectural Layout & Result Graph Models

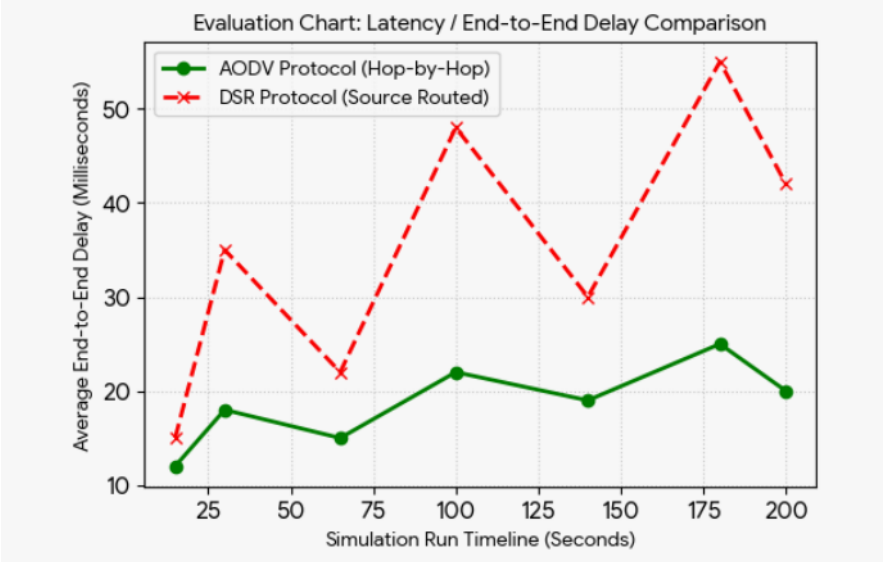
A. Metrics Collection Framework Flowchart

This block diagram outlines the data collection workflow used to capture network metrics for evaluation:



B. Expected Metrics Comparison Chart

This plot illustrates a typical end-to-end delay comparison chart generated by the evaluation software, highlighting the performance differences between the protocols over time:



Practical No. 05

Aim: Simulation of CSMA/CA MAC Protocol: Implement CSMA/CA protocol simulation and evaluate collision avoidance mechanisms.

Software Requirements

- **Simulation Framework:** OMNeT++ (v6.0 or later)
- **Model Library:** INET Framework (v4.4 or later)
- **Operating System:** Windows, Linux, or macOS

Theoretical Foundations

In shared wireless communication channels, multiple transmitter nodes frequently try to access the medium simultaneously. Unlike wired networks that can detect collisions in real-time (CSMA/CD), wireless nodes cannot reliably transmit and listen on the same frequency band at the same time. WSN and MANET nodes use the **CSMA/CA** MAC protocol to manage channel access.

Core Execution Stages of the Protocol:

1. **Carrier Sensing (CCA - Clear Channel Assessment):** Before initiating data transmission, a node monitors the physical medium to check if any active RF transmission energy is present.
2. **Inter-Frame Spacing (IFS):** If the channel is sensed as idle, the node pauses for a fixed duration called the **DIFS** (Distributed Inter-Frame Space) to ensure the medium is stable.
3. **Random Backoff Window:** If the channel is busy during sensing, or immediately after a successful transmission to prevent channel hogging, the node chooses a random backoff integer from a specific **Contention Window (CW)** range: \(\text{Backoff\ Time}=\text{Random}()\times \text{SlotTime}\) The node decrements this backoff counter only while the medium remains idle. If the channel becomes busy, the counter pauses until the channel clears again.
4. **Positive Acknowledgement (ACK):** Upon receiving a valid data frame, the destination node waits for a short **SIFS** interval and transmits an ACK frame back to the sender. If the sender does not receive this ACK within a timeout window, it assumes a collision occurred, increases its contention window size, and schedules a retransmission.

Network Configuration Files

Create the following files inside an empty project folder within your OMNeT++ workspace to construct the CSMA/CA simulation testbed.

File 1: Network Schema Definition (CsmaCaNetwork.ned)

This file instantiates the network topology, defining an environment where three nodes share a common wireless transmission space.

ned file

```
package csmaca_sim;

import inet.networklayer.configurator.ipv4.Ipv4NetworkConfigurator;
import inet.node.inet.WirelessHost;
import inet.physicallayer.wireless.common.contract.packetlevel.IRadioMedium;
import inet.visualizer.common.IntegratedVisualizer;

network CsmaCaNetwork
```

```
{
   parameters:
      @display("bgb=700,500;bgc=white");
   submodules:
      visualizer: IntegratedVisualizer {
         @display("p=100,50");
      }
      configurator: Ipv4NetworkConfigurator {
         @display("p=250,50");
      }
      radioMedium: <default("Ieee80211ScalarRadioMedium")> like IRadioMedium {
         @display("p=400,50");
      }
      // Deploy 3 wireless hosts sharing the same channel space
      host[3]: WirelessHost {
         @display("p=200,250,m,3,150,150");
      }
}
```

File 2: Simulator Configuration Script (omnetpp.ini)

This configuration file disables higher-level multi-hop routing layer protocols to isolate and evaluate raw MAC layer behaviors. It configures the nodes to use **IEEE 802.11 CSMA/CA** protocols while modifying frame parameters to capture clear contention trends.

ini file

```
[General]
network = CsmaCaNetwork
sim-time-limit = 100s

# --- 1. IP Stack and Address Assignment ---
*.configurator.assignAddresses = true

# --- 2. Isolation of the MAC Layer (Direct Local Connections) ---
# host[0] and host[1] send data frames directly to host[2] (Contention Scenario)
*.host[0].numApps = 1
*.host[0].app[0].typename = "UdpBasicApp"
*.host[0].app[0].destAddresses = "host[2]"
*.host[0].app[0].destPort = 5000
*.host[0].app[0].messageLength = 1000B
*.host[0].app[0].sendInterval = exponential(0.05s) # High-load contention traffic
*.host[0].app[0].startTime = 5s

*.host[1].numApps = 1
*.host[1].app[0].typename = "UdpBasicApp"
*.host[1].app[0].destAddresses = "host[2]"
*.host[1].app[0].destPort = 5000
*.host[1].app[0].messageLength = 1000B
*.host[1].app[0].sendInterval = exponential(0.05s)
*.host[1].app[0].startTime = 5s
```

```
# host[2] acts as a central sink receiving both concurrent data streams
*.host[2].numApps = 1
*.host[2].app[0].typename = "UdpSink"
*.host[2].app[0].localPort = 5000

# --- 3. CSMA/CA Protocol Parameters (802.11 NIC Setup) ---
*.host[*].wlan[*].typename = "Ieee80211Interface"
*.host[*].wlan[*].mac.typename = "Ieee80211Mac"
*.host[*].wlan[*].mac.dcf.channelAccess.cwMin = 31     # Minimum Contention Window
*.host[*].wlan[*].mac.dcf.channelAccess.cwMax = 1023       # Maximum Contention Window Size

# --- 4. Physical Layer and Radio Channel Range Setup ---
*.host[*].wlan[*].radio.typename = "Ieee80211ScalarRadio"
*.host[*].wlan[*].radio.transmitter.communicationRange = 300m

# --- 5. GUI Animation View settings ---
*.visualizer.infoVisualizer.displayInfos = true
*.visualizer.physicalLinkVisualizer.displayLinks = true
```

Operational Lab Execution Steps

1. Launch the OMNeT++ IDE environment and confirm that the baseline INET Framework component suite is built and active.
2. Create an assignment workspace project directory structure named csma_ca_simulation.
3. Save the CsmaCaNetwork.ned file mapping schema and the tracking script parameters inside omnetpp.ini.
4. Right-click the omnetpp.ini module structure and choose **Run As -> OMNeT++ Simulation**.
5. Inside the graphical user panel dashboard runtime canvas, hit the **Run** controls.
6. Open the graphical layout window for **host[0]** or **host[1]**, right-click the internal network interface card (NIC), and choose **Inspect Component** to observe backoff integer counters decremented in real-time.

Expected Analysis & Conclusion Profiles

- **Carrier Sensing and Backoff Observation:** When host[0] begins transmitting a data packet, the radioMedium module marks the shared channel status as busy. If host[1] generates an application frame during this exact interval, it performs carrier sensing, detects the active transmission energy, pauses its transmission, and chooses a random backoff integer value (e.g., 24 slots). You can see this value count down to zero inside the inspector view window once host[0] completes its transmission.
- **Collision Avoidance Assessment:** When host[0] and host[1] select different random backoff intervals, the node with the smaller backoff counter finishes counting down first and wins access to the medium. The node with the larger counter pauses its countdown, preventing an on-air packet collision. If both nodes choose the exact same backoff number, an on-air collision occurs at the receiver node (host[2]), which drops the corrupted packet frames and does not return an ACK message. The sending nodes then detect this missing

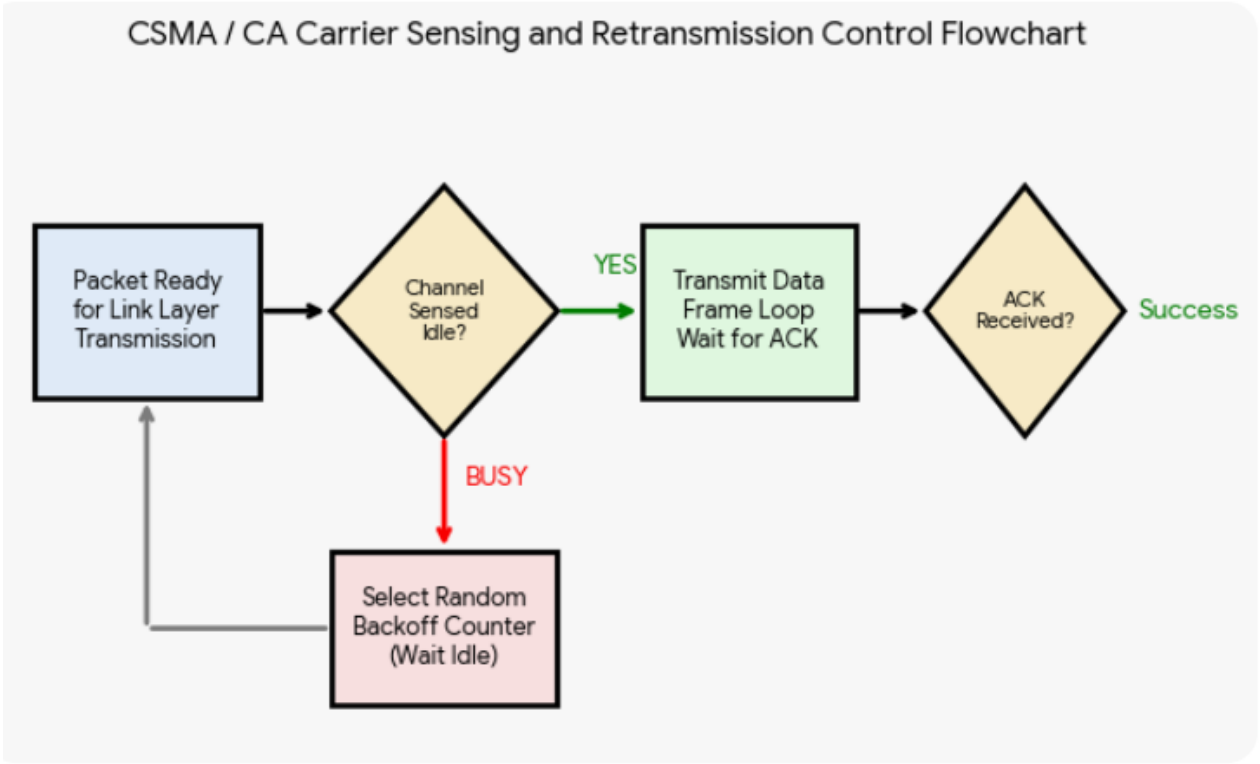
ACK timeout, double their contention window size (e.g., from 31 to 63), and pick a new backoff number to resolve the conflict.

- **Conclusion:** This practical successfully demonstrates the execution properties of the CSMA/CA protocol. The simulation effectively illustrates how random backoff intervals and carrier sensing mechanisms prevent wireless data collisions in shared channel environments.

Behavioral Flowcharts & Layout Diagrams

A. CSMA/CA Logic Processing Decisions Sequence

The flowchart below maps out the sequence of steps the MAC layer follows to process and transmit data frames while avoiding on-air collisions:



Practical No. 06

Aim: Simulation of TDMA-based MAC Protocol: Simulate TDMA slot allocation and compare energy efficiency with CSMA.

Software Requirements

- **Simulation Framework:** OMNeT++ (v6.0 or later)
- **Model Library:** INET Framework (v4.4 or later)
- **Operating System:** Windows, Linux, or macOS

Theoretical Foundations

Unlike contention-based access methods (CSMA/CA) where nodes compete for the wireless channel, **TDMA** is a collision-free, schedule-based MAC protocol. The timeline is partitioned into structural frames, and each frame is subdivided into a fixed sequence of cyclic **time slots**.

Core TDMA Mechanics:

1. **Deterministic Slot Assignment:** Each sensor node is assigned a specific, dedicated slot within the frame. A node can transmit *only* during its designated interval, entirely avoiding data packet collisions.
2. **Duty Cycling and Power States:** During slots assigned to other nodes, a TDMA node can completely turn off its wireless transceiver and enter a **low-power sleep state**.
3. **Primary Energy Consumption Bottlenecks in WSNs:**
   - **Collisions:** Energy wasted retransmitting packets corrupted by overlapping signals (Mitigated by TDMA).
   - **Idle Listening:** Energy wasted listening to an empty channel waiting for data (Eliminated by TDMA via scheduled sleep).
   - **Overhearing:** Energy wasted receiving packets intended for other nodes (Eliminated by TDMA slot isolation).

Energy Equations:

The consumption profile metrics for the network interfaces are calculated as follows:

$$E_{\text{total}} = (P_{\text{tx}} \cdot t_{\text{tx}}) + (P_{\text{rx}} \cdot t_{\text{rx}}) + (P_{\text{idle}} \cdot t_{\text{idle}}) + (P_{\text{sleep}} \cdot t_{\text{sleep}})$$

Network Configuration Files

Create the following files inside your OMNeT++ project directory to construct the TDMA vs CSMA testing environment.

File 1: Network Schema Definition (TdmaNetwork.ned)

This file instantiates the network topology, deploying four wireless sensor hosts and an explicit energy storage tracker submodule configuration inside each node.

ned file

```
package tdma_sim;

import inet.networklayer.configurator.ipv4.Ipv4NetworkConfigurator;
import inet.node.inet.WirelessHost;
import inet.physicallayer.wireless.common.contract.packetlevel.IRadioMedium;
import inet.visualizer.common.IntegratedVisualizer;

network TdmaNetwork
```

```
{
    parameters:
        @display("bgb=750,550;bgc=white");
    submodules:
        visualizer: IntegratedVisualizer {
            @display("p=100,50");
        }
        configurator: Ipv4NetworkConfigurator {
            @display("p=250,50");
        }
        radioMedium: <default("Ieee802154ScalarRadioMedium")> like IRadioMedium {
            @display("p=400,50");
        }
        // Deploy a cluster of 4 nodes with energy consumers integrated
        host: WirelessHost {
            @display("p=200,250,m,4,150,150");
        }
}
```

File 2: Simulator Configuration Script (omnetpp.ini)

This initialization profile sets up the **INET energy model library**, sets specific radio consumption current parameters, and maps out the TDMA timing structures.

ini file

```
[General]
network = TdmaNetwork
sim-time-limit = 150s

# --- 1. Address Configurator ---
*.configurator.assignAddresses = true

# --- 2. Application Layer Traffic Generating Matrix ---
# Nodes 0, 1, and 2 send periodic sensor telemetry to Central Base Station (Node 3)
*.host[0..2].numApps = 1
*.host[0..2].app.typename = "UdpBasicApp"
*.host[0..2].app.destAddresses = "host[3]"
*.host[0..2].app.destPort = 8000
*.host[0..2].app.messageLength = 64B
*.host[0..2].app.sendInterval = 1s
*.host[0..2].app.startTime = 5s

*.host[3].numApps = 1
*.host[3].app.typename = "UdpSink"
*.host[3].app.localPort = 8000

# --- 3. Hardware Energy Consumer Module Settings ---
*.host[*].energyStorage.typename = "SimpleCcEnergyStorage"
*.host[*].energyStorage.nominalCapacity = 1000J
*.host[*].energyStorage.initialCapacity = 1000J
```

```
# Radio power draws (expressed in mA) matching sensor node specs
*.host[*].wlan[*].radio.energyConsumer.typename = "StateBasedEpEnergyConsumer"
*.host[*].wlan[*].radio.energyConsumer.transmitterPowerConsumption = 25mW
*.host[*].wlan[*].radio.energyConsumer.receiverPowerConsumption = 20mW
*.host[*].wlan[*].radio.energyConsumer.idlePowerConsumption = 15mW
*.host[*].wlan[*].radio.energyConsumer.sleepPowerConsumption = 0.01mW

#
============================================================
# COMPETING RUN MODULE CONFIGURATIONS
#
============================================================
[Config TDMA_Run]
description = "Collision-Free TDMA Slot Schedule Implementation"
*.host[*].wlan[*].typename = "Ieee802154NarrowbandInterface"
*.host[*].wlan[*].mac.typename = "Ieee802154TdmaMac"
*.host[*].wlan[*].mac.frameLength = 4s
*.host[*].wlan[*].mac.slotDuration = 1s
# Assign explicit slot arrays: Node 0 -> Slot 0, Node 1 -> Slot 1, Node 2 -> Slot 2...
*.host[0].wlan[*].mac.slotNumber = 0
*.host[1].wlan[*].mac.slotNumber = 1
*.host[2].wlan[*].mac.slotNumber = 2
*.host[3].wlan[*].mac.slotNumber = 3

[Config CSMA_Run]
description = "Contention-Based CSMA/CA MAC Baseline"
*.host[*].wlan[*].typename = "Ieee802154NarrowbandInterface"
*.host[*].wlan[*].mac.typename = "Ieee802154CsmaCaMac"
```

Operational Lab Analysis Steps

1. Initialize the OMNeT++ workspace framework environment. Create a folder named tdma_vs_csma.
2. Save the structure fields inside TdmaNetwork.ned and the runtime setups inside omnetpp.ini.
3. Launch the simulation launcher tool configuration. Execute **TDMA_Run** completely, then run **CSMA_Run** to generate separate data vectors.
4. Open the analysis file suite tool workspace (.anf), import the results files, and select the variable line: *.host[*].energyStorage.residualCapacity.
5. Plot the remaining energy levels for both configurations on a single line chart to evaluate and compare their overall efficiency.

MAC Protocol Performance Comparison Matrix

The table below outlines the trade-offs observed between schedule-based (TDMA) and contention-based (CSMA) MAC layers under periodic WSN traffic loads:

Evaluation Performance Metric	**Schedule-Based MAC (TDMA)**	**Contention-Based MAC (CSMA)**

Packet Collisions Rate	**0% (Zero)**: Slots isolate transmissions.	Occurs frequently during synchronized traffic spikes.
Transceiver Idle Listening	None: Nodes sleep during unassigned slots.	High: Transceivers remain continuously active.
Energy Dissipation Profile	**Minimal**: Power drains slowly due to duty cycling.	**High**: Continuous idle listening quickly drains battery.
Channel Capacity Utilization	Highly efficient under heavy, constant traffic loads.	Suffers from backoff overhead under heavy traffic.
Adaptability to Network Changes	Rigid: Requires rescheduling if a node drops out.	Highly flexible: Nodes join or leave dynamically.
Synchronization Overhead	Requires strict, continuous time synchronization.	Completely asynchronous (No sync overhead).

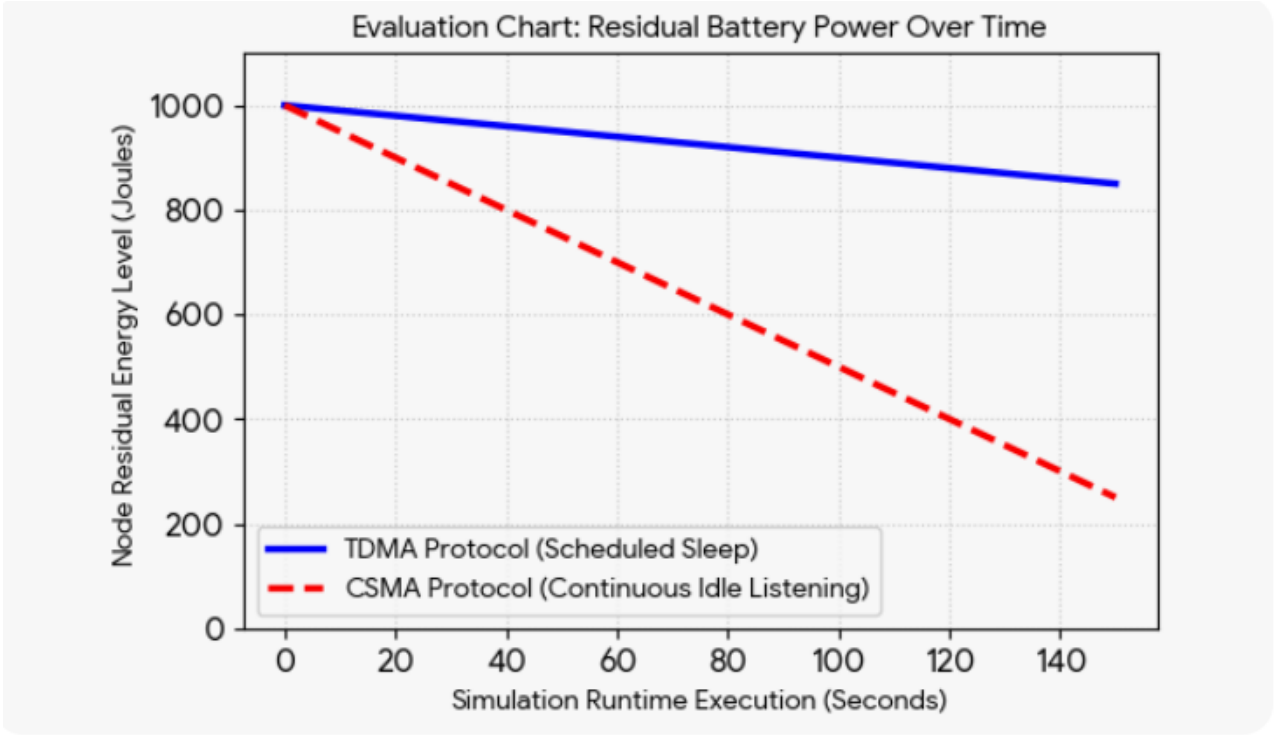
Schematic Illustrations & Flowcharts

A. TDMA Cyclic Timing Frame Matrix Structure

This timeline schematic illustrates how the TDMA frame divides channel access into deterministic slots, allowing nodes to cycle between active transmission and low-power sleep states:

B. Comparative Battery Consumption Profile Chart

This plot illustrates the remaining energy over time, highlighting how TDMA's schedule-based duty cycling preserves battery life compared to CSMA's continuous idle listening:



Practical No. 07

Aim: Energy-Aware MAC Protocol for WSN: Implement duty-cycling MAC protocol and evaluate energy conservation in sensor networks.

Software Requirements

- **Simulation Engine:** OMNeT++ (v6.0 or later)
- **Model Component Library:** INET Framework (v4.4 or later)
- **Operating System:** Windows, Linux, or macOS

Theoretical Foundations

Energy conservation is the most critical constraint in Wireless Sensor Networks (WSNs) because sensor nodes are typically battery-powered and deployed in inaccessible environments. The radio transceiver is the primary consumer of energy.

Key Sources of Radio Energy Waste:

1. **Idle Listening:** The radio remains active in receive mode waiting for data that rarely arrives. This is the largest source of waste in low-traffic settings.
2. **Overhearing:** A node consumes energy receiving and decoding packets destined for other nodes.
3. **Collisions:** Wasted energy when overlapping packets corrupt each other on-air and require retransmissions.

Duty-Cycling Operational Strategy (e.g., S-MAC):

To mitigate idle listening, duty-cycling protocols periodically force the radio transceiver into a low-power sleep state. Time is divided into frames containing a fixed **Active Period** (for synchronization and data exchange) and a **Sleep Period** (where the radio is shut down).

The **Duty Cycle (\(\mathbf{D}\))** is expressed as the percentage of active time relative to total frame time: \(D=\frac{T_{\text{active}}}{T_{\text{active}}+T_{\text{sleep}}}\times 100\%\)

Network Configuration Files

Create the following files inside your OMNeT++ project directory to configure the energy-aware duty-cycling comparison layout.

File 1: Network Schema Definition (EnergyAwareNetwork.ned)

This file instantiates the network topology, adding four wireless sensor nodes equipped with integrated power measurement systems.

ned file

```
package energy_mac_sim;

import inet.networklayer.configurator.ipv4.Ipv4NetworkConfigurator;
import inet.node.inet.WirelessHost;
import inet.physicallayer.wireless.common.contract.packetlevel.IRadioMedium;
import inet.visualizer.common.IntegratedVisualizer;

network EnergyAwareNetwork
{
    parameters:
        @display("bgb=750,550;bgc=white");
    submodules:
```

```
        visualizer: IntegratedVisualizer {
            @display("p=100,50");
        }
        configurator: Ipv4NetworkConfigurator {
            @display("p=250,50");
        }
        radioMedium: <default("Ieee802154ScalarRadioMedium")> like IRadioMedium {
            @display("p=400,50");
        }
        // Instantiates a 4-node tracking mesh group
        host: WirelessHost {
            @display("p=200,250,m,4,150,150");
        }
}
```

File 2: Simulator Configuration Script (omnetpp.ini)
This script isolates the MAC layer, sets up low-traffic workloads, handles INET battery capacity modeling parameters, and implements inherited profiles for comparing duty-cycling vs. standard MAC protocols.

ini file

```
[General]
network = EnergyAwareNetwork
sim-time-limit = 200s

# --- 1. Network Address Configuration ---
*.configurator.assignAddresses = true

# --- 2. Low-Traffic Application Workload Setup ---
# Sensor nodes 0, 1, and 2 transmit readings to Base Station node 3
*.host[0..2].numApps = 1
*.host[0..2].app.typename = "UdpBasicApp"
*.host[0..2].app.destAddresses = "host"
*.host[0..2].app.destPort = 9000
*.host[0..2].app.messageLength = 32B
*.host[0..2].app.sendInterval = 10s   # Periodic low data rate traffic
*.host[0..2].app.startTime = 5s

*.host.numApps = 1
*.host.app.typename = "UdpSink"
*.host.app.localPort = 9000

# --- 3. Hardware Energy & Battery Tracking Engine Setup ---
*.host[*].energyStorage.typename = "SimpleCcEnergyStorage"
*.host[*].energyStorage.nominalCapacity = 500J
*.host[*].energyStorage.initialCapacity = 500J

# Node radio active current consumption profiles
```

```
*.host[*].wlan[*].radio.energyConsumer.typename = "StateBasedEpEnergyConsumer"
*.host[*].wlan[*].radio.energyConsumer.transmitterPowerConsumption = 30mW
*.host[*].wlan[*].radio.energyConsumer.receiverPowerConsumption = 25mW
*.host[*].wlan[*].radio.energyConsumer.idlePowerConsumption = 20mW
*.host[*].wlan[*].radio.energyConsumer.sleepPowerConsumption = 0.02mW

#
============================================================
# COMPETING SIMULATION PROFILES FOR ENERGY ANALYSIS
#
============================================================
[Config DutyCycling_MAC]
description = "Energy-Aware Sensor-MAC (S-MAC) Schedule Protocol"
*.host[*].wlan[*].typename = "Ieee802154NarrowbandInterface"
*.host[*].wlan[*].mac.typename = "Ieee802154SensorMac"   # INET S-MAC Component
*.host[*].wlan[*].mac.slotDuration = 1s
*.host[*].wlan[*].mac.dutyCycle = 10%                    # 10% Active, 90% Sleep state

[Config Baseline_AlwaysOn_MAC]
description = "Standard Un-cycled Always-On CSMA/CA Protocol Baseline"
*.host[*].wlan[*].typename = "Ieee802154NarrowbandInterface"
*.host[*].wlan[*].mac.typename = "Ieee802154CsmaCaMac"   # Stays continuously in idle listen
```

Operational Lab Execution Steps

1. Launch the OMNeT++ IDE workspace and ensure the INET Framework baseline is active.
2. Create an assignment workspace project directory structure named energy_mac_simulation.
3. Save the module mappings into EnergyAwareNetwork.ned and the execution rules within omnetpp.ini.
4. Run the **DutyCycling_MAC** profile first to completion, then run the **Baseline_AlwaysOn_MAC** scenario.
5. Open the built-in Analysis Tool (.anf), target the residualCapacity variables, and plot them together to view the battery consumption curves.

Quantitative Energy Management Comparison Matrix

The table below outlines the trade-offs observed between energy-aware duty-cycling MAC layers and standard un-cycled configurations under light traffic loads:

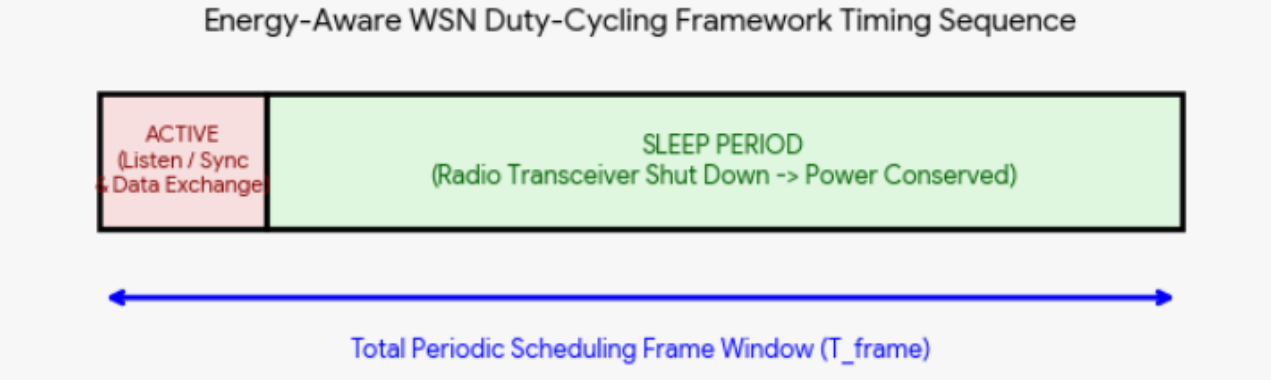
Evaluation Metric Indicator	**Energy-Aware Duty-Cycling MAC (S-MAC)**	**Baseline Always-On MAC (CSMA/CA)**
Radio Operating State Mode	Periodically cycles between active listen and low-power sleep.	Remains permanently in active listen mode when not transmitting.
Idle Listening Mitigation	**High**: Shuts down the radio for up to 90% of the frame duration.	**None**: Continually monitors the empty channel, draining power.
Average Current Draw Profile	**Extremely Low**: Mostly draws minimal sleep-state power.	**High**: Drains battery steadily at full idle listening power levels.

Packet Delivery Latency	**Higher**: Packets are buffered until the next scheduled active slot.	**Minimal**: Packets transmit instantly across the active medium.
Network Battery Lifespan	**Extended**: Significantly prolongs operational lifetime.	**Short**: Nodes exhaust their battery quickly due to idle listening.

Architectural Flowcharts & Evaluation Models

A. Duty-Cycling Active vs. Sleep Timing Framework

This timeline diagram illustrates the periodic duty cycle, showing how nodes switch between brief active periods for synchronization and data exchange and long sleep periods for energy conservation:



Practical No. 08

Aim: MANET Simulation with Directional Antenna: Simulate MANET using directional antennas and analyze improvements in spatial reuse and interference reduction.

Mobile Ad Hoc Networks (MANETs) are self-configuring, infrastructure-less networks in which mobile nodes communicate wirelessly without relying on fixed infrastructure. These networks are commonly used in military operations, disaster recovery scenarios, and vehicular communication systems because of their dynamic topology and decentralized nature.

• In traditional MANETs, communication typically relies on omnidirectional antennas that transmit signals uniformly in all directions. While simple to implement, this approach leads to high levels of interference, increased energy consumption, and reduced overall network efficiency due to unnecessary signal propagation.

• To address these limitations, directional antennas have been introduced into MANET environments and simulations. Unlike omnidirectional antennas, directional antennas concentrate signal transmission in a specific direction, allowing more focused and efficient communication between nodes.

• The use of directional antennas improves communication range while significantly reducing interference and packet collisions. This focused transmission increases spatial reuse, enhances network throughput, and lowers energy consumption, thereby making MANETs more efficient and reliable.

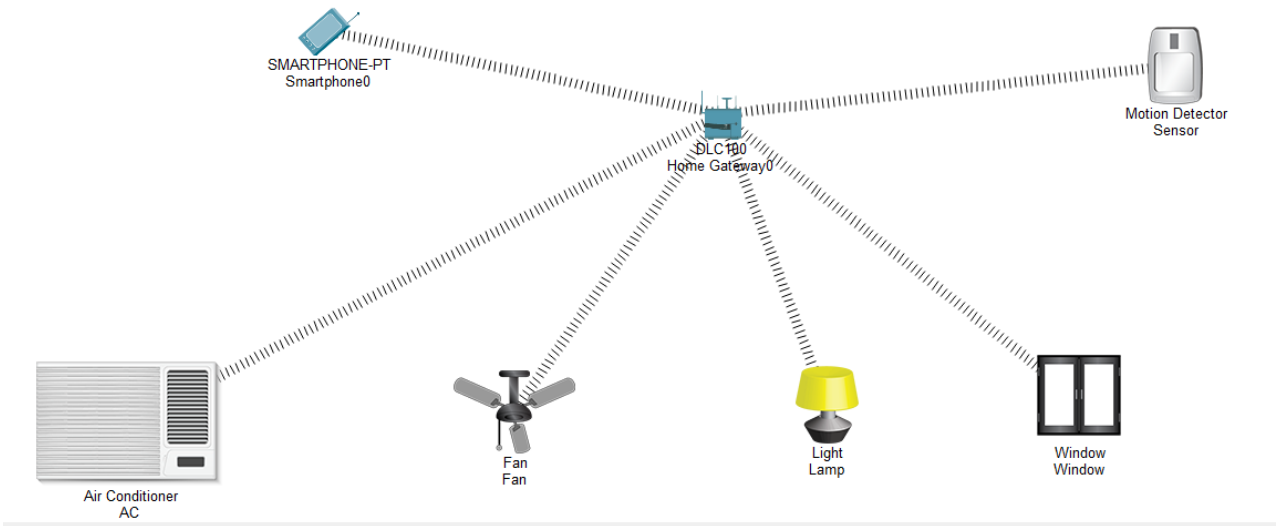
Simulating MANETs with directional antennas requires accurate modeling of various antenna types, including switched beam antennas, sector antennas, and adaptive array antennas. These models help represent real-world directional communication behavior within the network.

Such simulations allow researchers to analyze key performance parameters such as network connectivity, signal strength, packet delivery ratio, and end-to-end delay. Evaluating these metrics provides insights into the effectiveness of directional antennas under different network conditions.

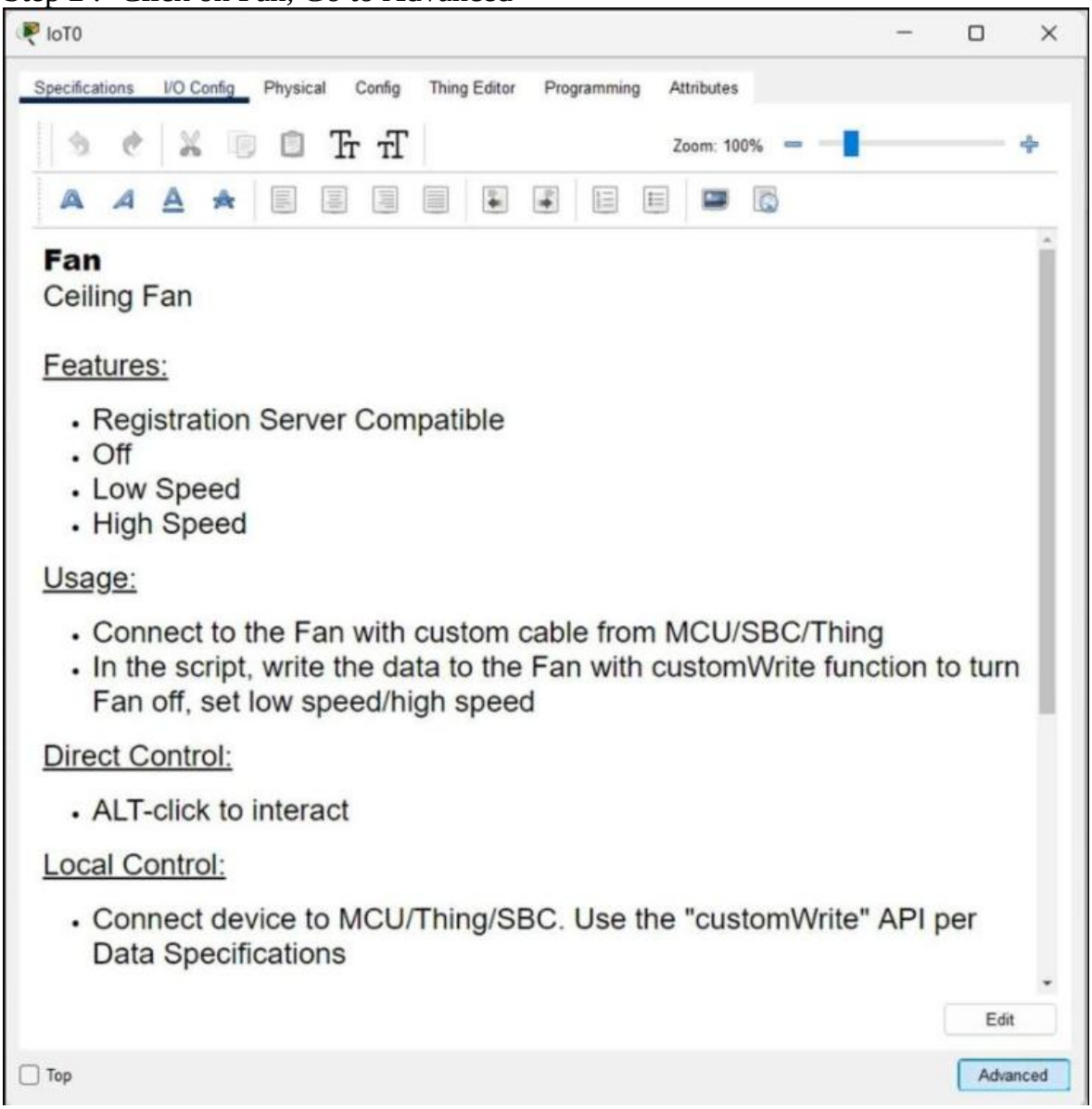
Advanced network simulation tools like NS-2, NS-3, and OMNeT++ are widely used to study the impact of directional antennas in MANET environments. These simulators support detailed modeling and performance analysis.

By integrating directional antennas into MANET simulations, researchers can explore new routing protocols, interference mitigation techniques, and energy-efficient communication strategies, ultimately improving network scalability, performance, and reliability.

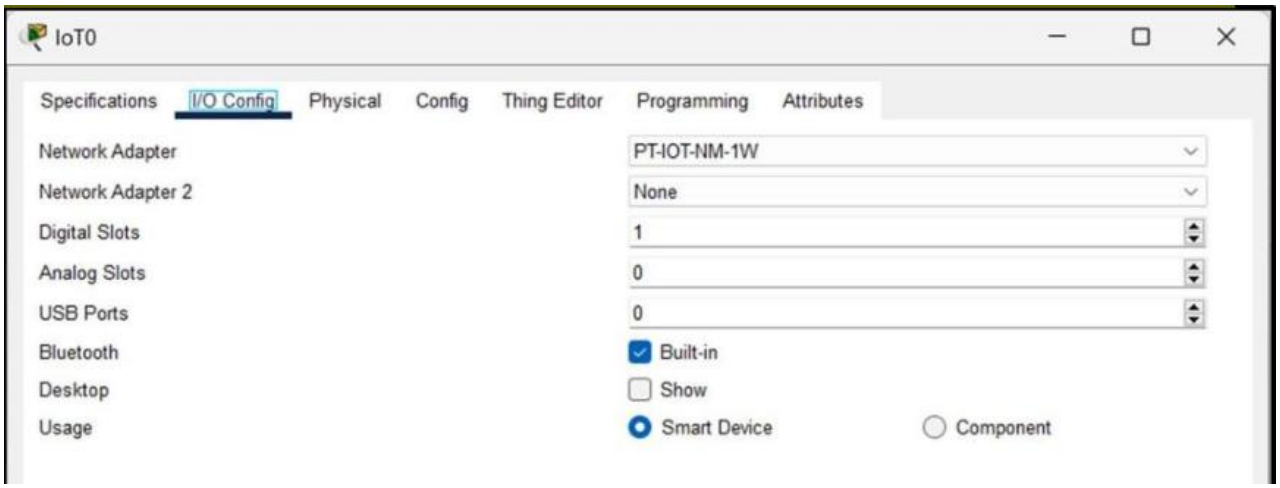
Step 1 :- From the Toolbar, Select 1 Fan, 1 Door, 1 Light, 1 Motion Detector, 1 Smartphone and 1 Home Gateway



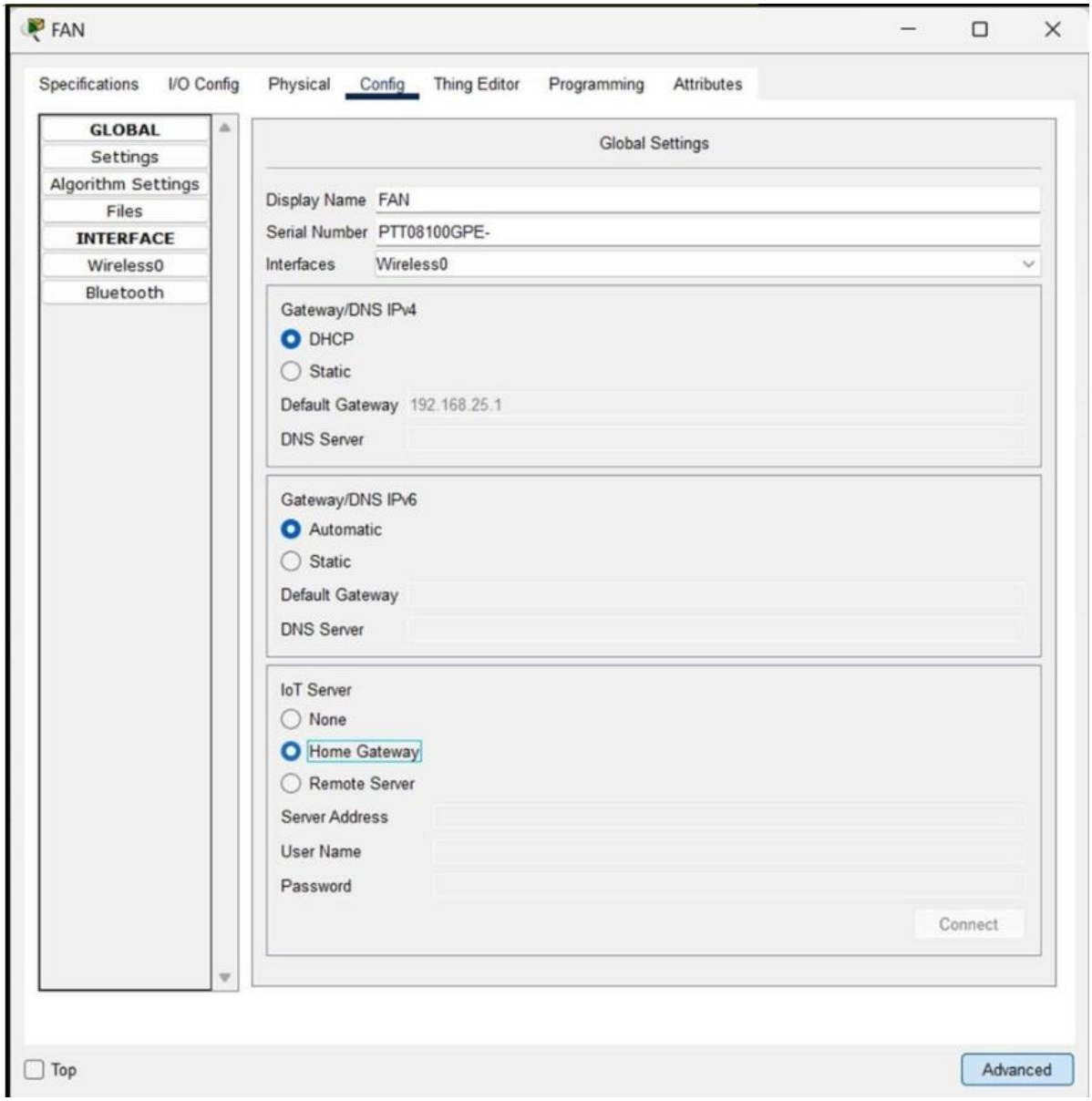
Step 2 :- Click on Fan, Go to Advanced



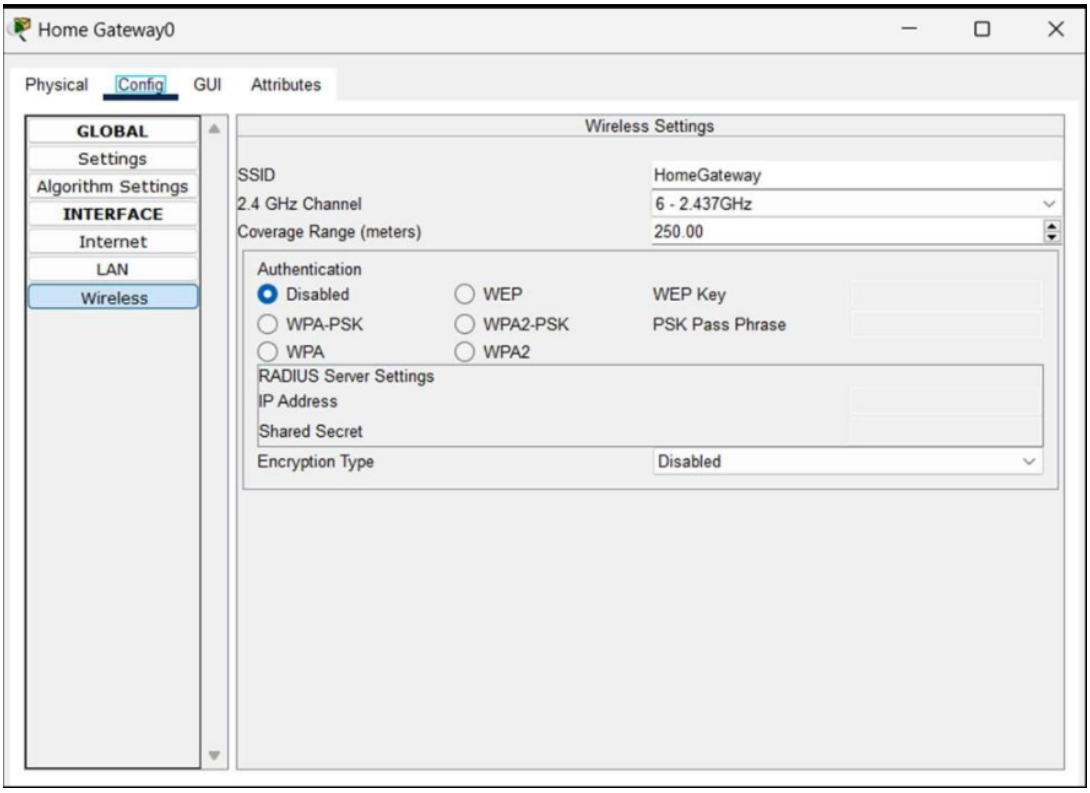
Step 3 :- Go to I/O Config, Change Network Adapter to “PT-IOT-NM-1W”



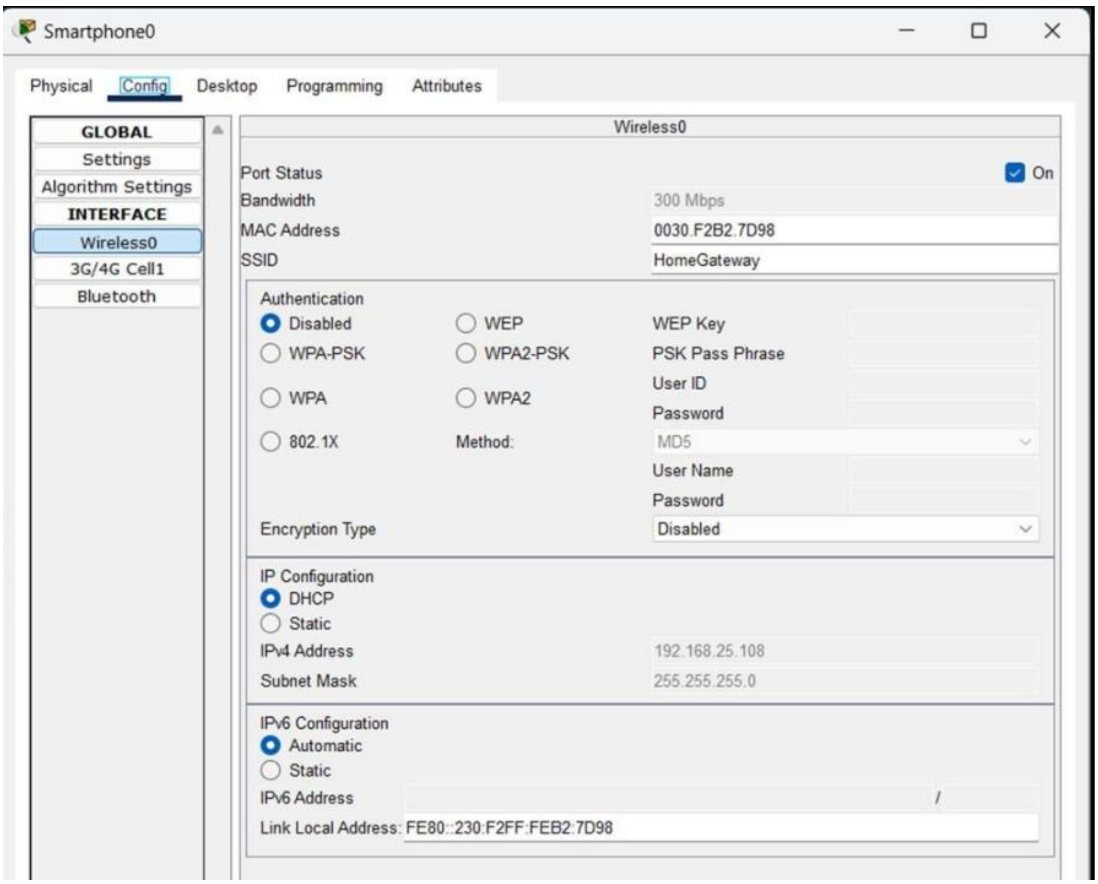
Step 4 :- Go to Config, in Iot Server select Home Gateway



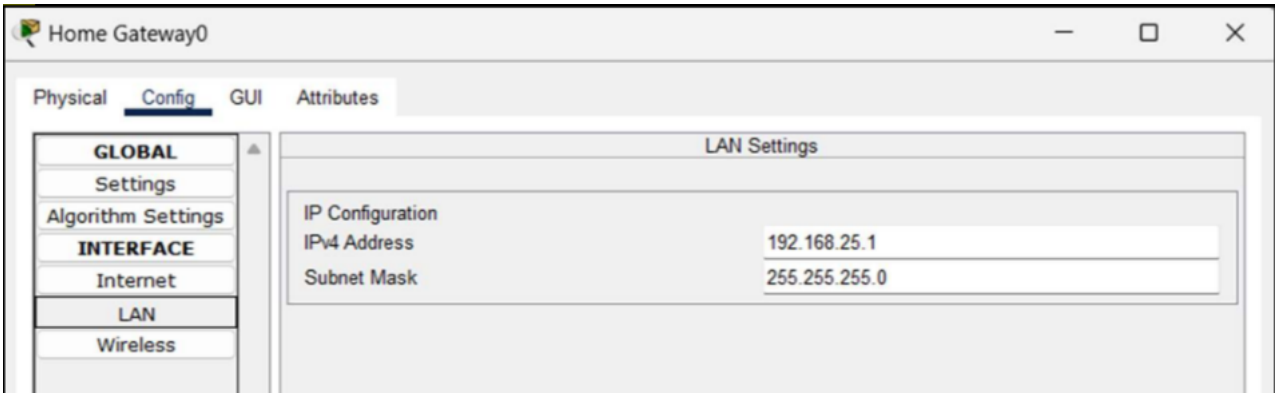
Step 5 :- In HomeGateway Go to Config tab, click on wireless from there copy the SSID of home gateway



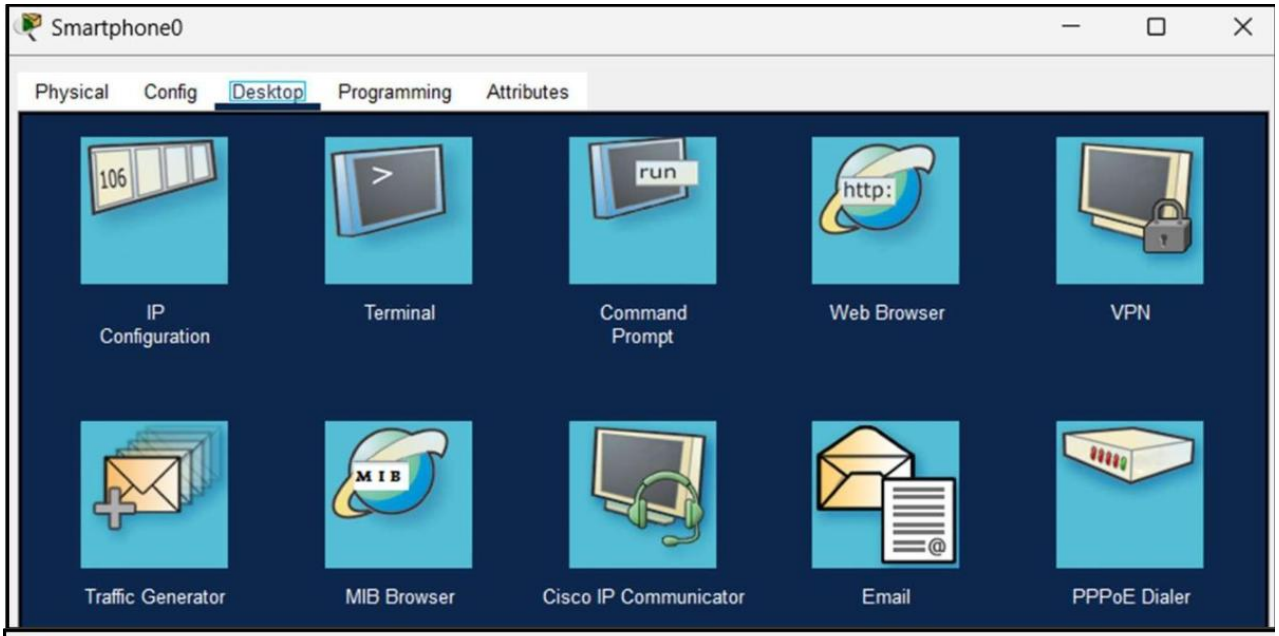
Step 6:- Now Click on Smartphone Go to Config Tab, click on Wireless0 Paste the SSID

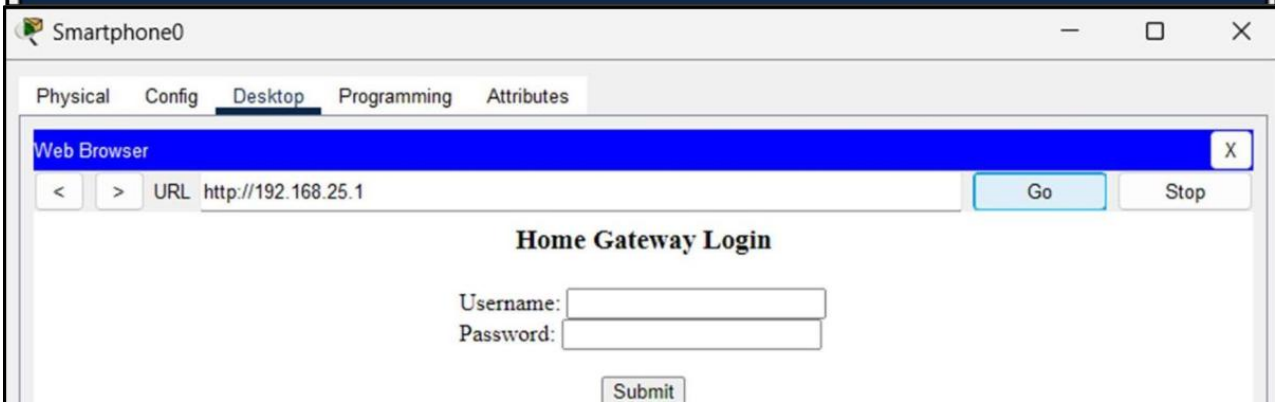


Step 7 :- In HomeGateway Go to Config tab, click on LAN from there copy the IP address of home gateway

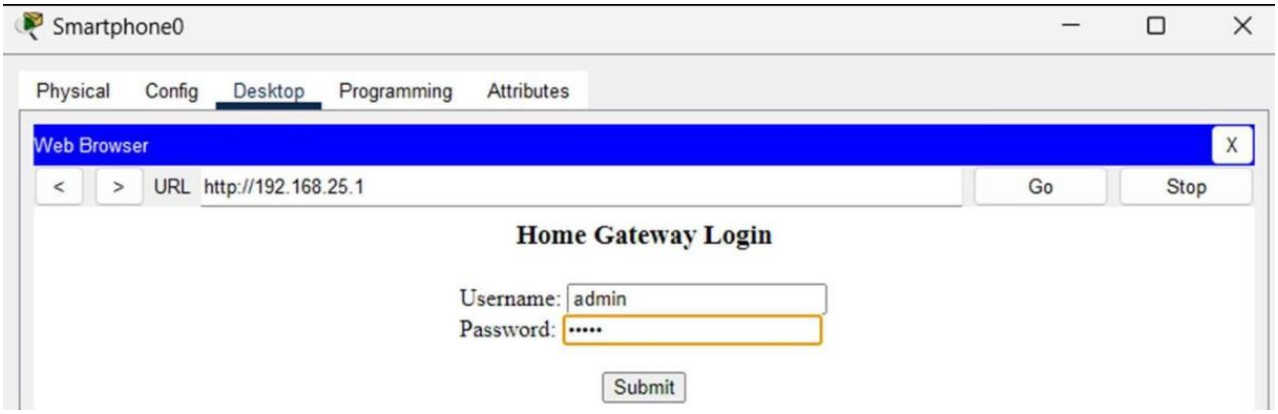


Step 8 :- Now Click on Smartphone Go to Desktop tab, click on web browser, paste the ip address and click on go

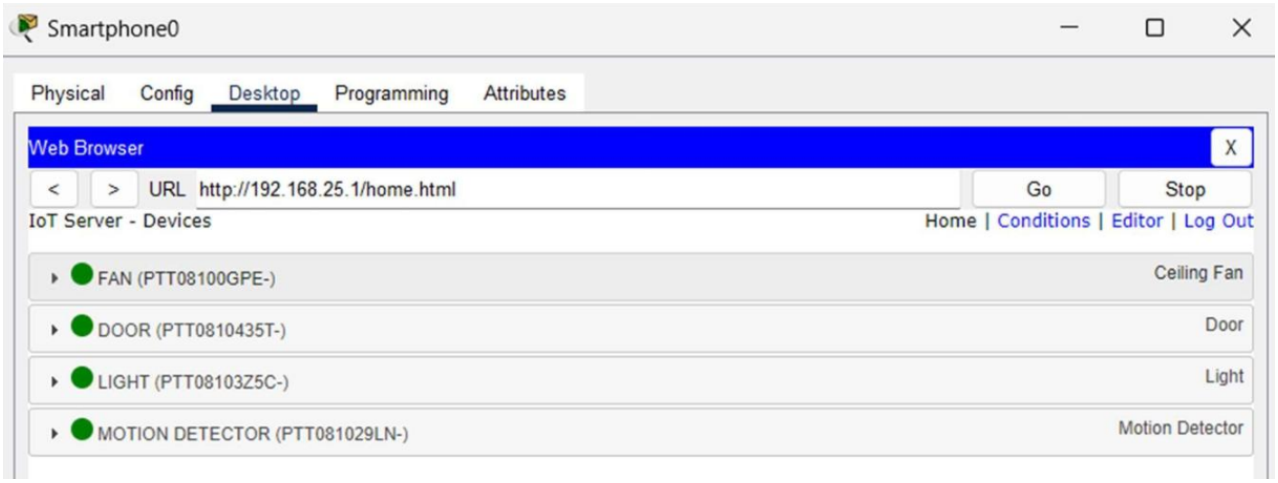




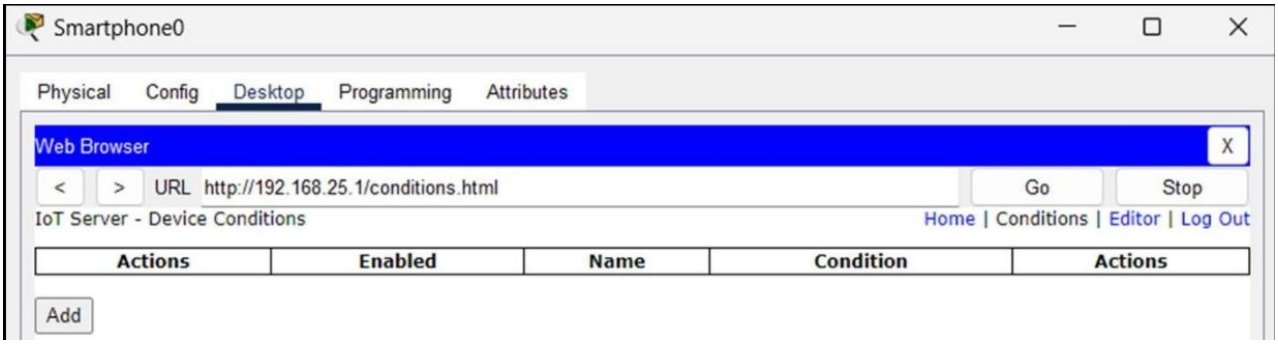
Step 9 :- for username type “admin” and for password type “admin” then click on submit



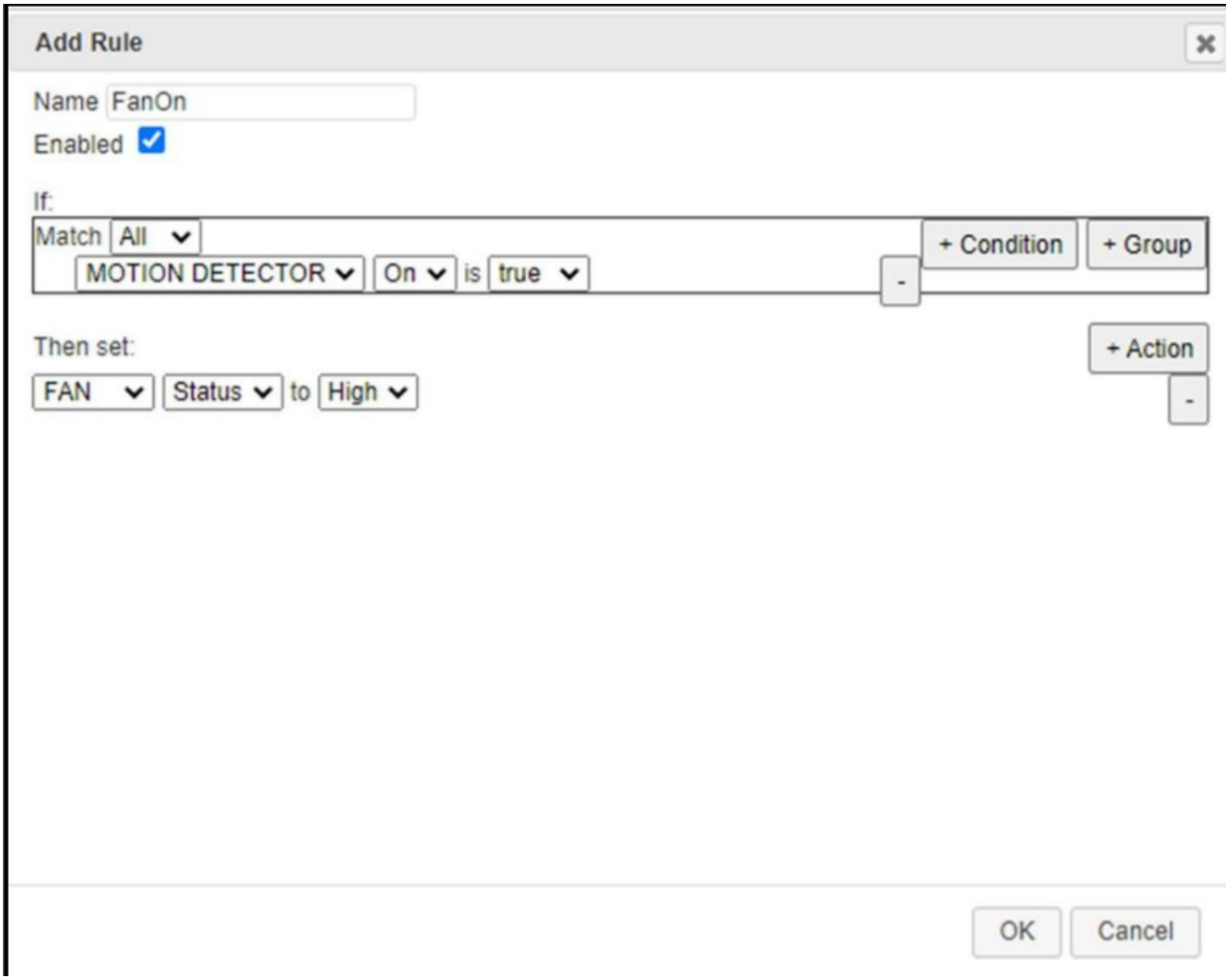
Step 10 :- Now Click on Conditions



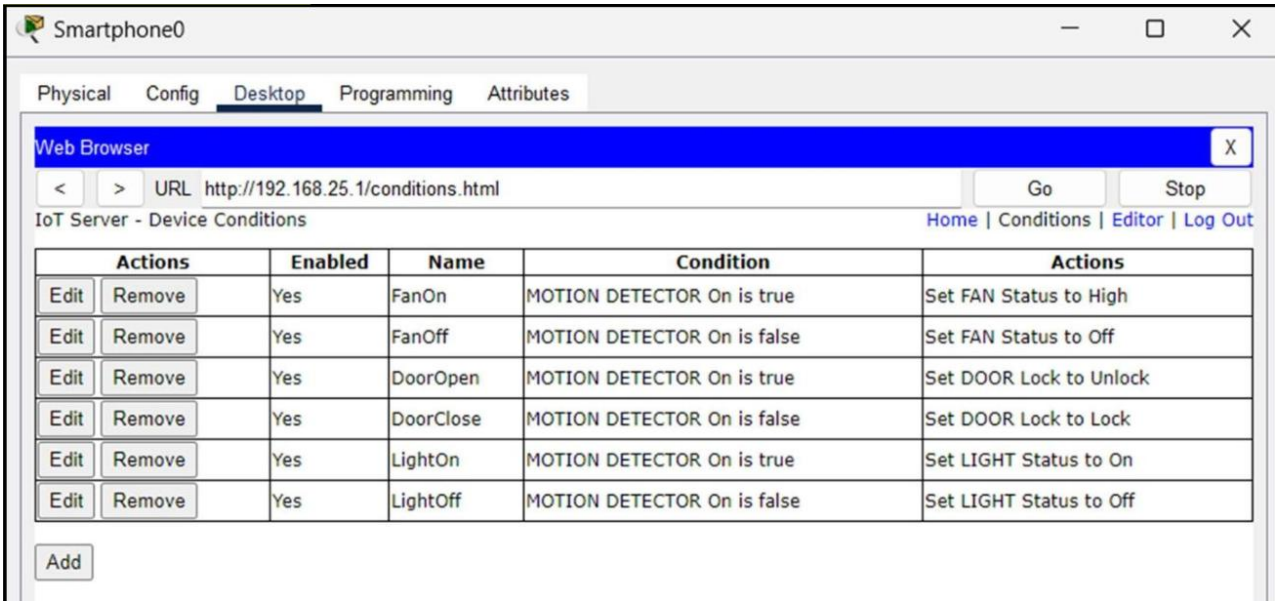
Step 11 :- Click on Add



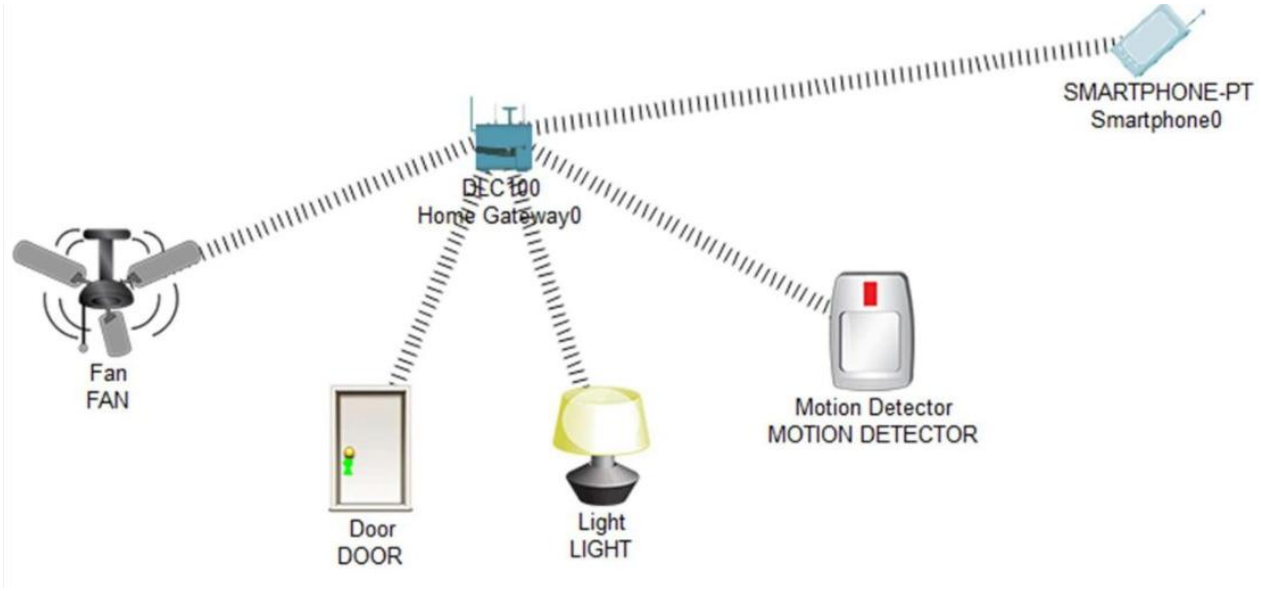
Step 12 :- In Add Rule Tab, Provide the Name for your rule. Specify the conditions and click on ok



Step 13 :- Add multiple conditions like this for FanOn, FanOff, LightOn, LightOff, DoorOpen, DoorClose



Step 14 :- Now hold Ctrl+Alt button, and hover the cursor on motion detector to activate. This will be the final result.



Practical No. 09

Aim: Wireless Sensor Network Deployment and Coverage Analysis: Simulate random and grid deployment of sensor nodes and evaluate network coverage and connectivity.

Software Requirements

- **Simulation Tool:** Cisco Packet Tracer (v8.0 or later recommended)
- **Operating System:** Windows, Linux, or macOS

Theoretical Foundations

Node deployment strategy directly affects the overall quality of service in a WSN. It influences key operational traits including deployment cost, sensory data coverage, and message forwarding connectivity.

Core Assessment Properties:

1. **Sensory Coverage:** The total geographical area monitored by the network. A physical target coordinate \((x, y)\) is covered if its Euclidean distance to at least one sensor node is less than or equal to the node's **sensing range (R_s)**.
2. **Network Connectivity:** The network's ability to forward data packets back to a central destination. Connectivity depends on whether nodes are within each other's **communication range (R_c)** enabling a multi-hop wireless path back to the sink node.
3. **Grid Deployment Strategy:** Nodes are placed at uniform intervals according to a precise geometric structure (e.g., square or hexagonal grid). This approach maximizes structural coverage and minimizes redundant overlapping zones, but it requires manual positioning that can be difficult in harsh environments.
4. **Random Deployment Strategy:** Nodes are distributed scattered across the environment (e.g., dropped from an aircraft). While cheap and fast to deploy, it frequently creates high-density clusters with overlapping coverage alongside dead zones that isolate parts of the network.

Practical Setup Framework inside Packet Tracer

To test and contrast both deployment configurations inside the Packet Tracer workspace canvas, use the structural component mappings below:

Network Components Setup:

- **Central Sink Node / Gateway: Home Gateway** or **Registration Server** (acting as the central WSN sink node).
- **WSN Sensor Motes: IoT Microcontroller (MCU-PT)** board units or specialized environmental sensors (e.g., temperature/smoke detectors) equipped with **Wireless Network Interface Cards (W-NIC)**

Detailed Configuration & Simulation Steps

Step 1: Implementing the Grid Deployment Blueprint

1. Open Cisco Packet Tracer and navigate to the **Logical Workspace**.
2. Place a **Home Gateway** (under Network Devices -> Wireless Devices) exactly in the center of the canvas area.
3. Deploy 8 **IoT sensor nodes** (under End Devices -> Industrial or Components) at regular, uniform distances surrounding the gateway to create a square matrix grid.

4. Open each sensor node's **Config** panel. Under the **Advanced** option, verify that the network interface card is set to a wireless module (e.g., PT-IOT-NM-1W supporting 2.4GHz links).
5. Configure each node to register automatically with the central Home Gateway using Link-Local IP schemes or DHCP options.
6. **Coverage Check:** Turn on **Wireless Coverage Clusters** in the options bar. Verify that the wireless coverage circles overlap uniformly without leaving empty gaps or isolated nodes.

Step 2: Implementing the Random Deployment Blueprint

1. Clear the canvas or create a separate, isolated segment on the right side of the workspace. Place a second Home Gateway.
2. Deploy 8 wireless sensor nodes scattered randomly across the canvas. Place some nodes very close together to form clusters, and place others far away near the edges of the screen.
3. Verify that all nodes use the same wireless network parameters.
4. **Connectivity Check:** Observe the automatic link status lines. Look for **isolated edge units (dead zones)** that cannot communicate with the gateway or neighboring relay nodes because they are too far away.

Step 3: Measuring Network Performance

1. Switch Packet Tracer from **Realtime Mode** to **Simulation Mode** (bottom right corner tabs).
2. Use the **Add Simple PDU tool (Envelope Icon)** to send test ping packets from each sensor node back to the central gateway node.
3. Click **Capture/Forward** to trace the packet pathways step-by-step through the network.
4. Record how many packets are successfully delivered and how many fail due to out-of-range link drops.

Quantitative Deployment Comparison Matrix

The table below outlines the trade-offs observed between Grid and Random WSN deployment strategies based on the Packet Tracer simulation metrics:

Network Evaluation Property	**Uniform Grid Deployment Profile**	**Random Scattering Deployment Profile**
Placement Accuracy	Deterministic (Strictly ordered positions).	Stochastic (Randomly scattered across space).
Area Sensing Coverage Efficiency	**High**: Minimal overlapping zones maximize total monitored area.	**Low**: High-density clusters leave blind spots elsewhere.
Link Connectivity Stability	**100% Stable**: Uniform distances ensure reliable communication links.	**Variable**: Isolated nodes can create disconnected sections.
Multi-Hop Path Efficiency	Predictable paths minimize routing hops and delays.	Unpredictable; redundant links can cause transmission loops.
Packet Delivery Success Rate	High; minimal packet drops across all nodes.	Lower; frequent packet drops at distant edge nodes.
Real-world Deployment Feasibility	Hard/Expensive (Requires manual positioning).	Easy/Cheap (Suitable for hazardous or remote areas).

Expected Analysis & Conclusion Profiles

- **Grid Topology Analysis Observations:** The grid simulation shows a balanced, well-distributed topology. Each sensor node successfully establishes a wireless connection link

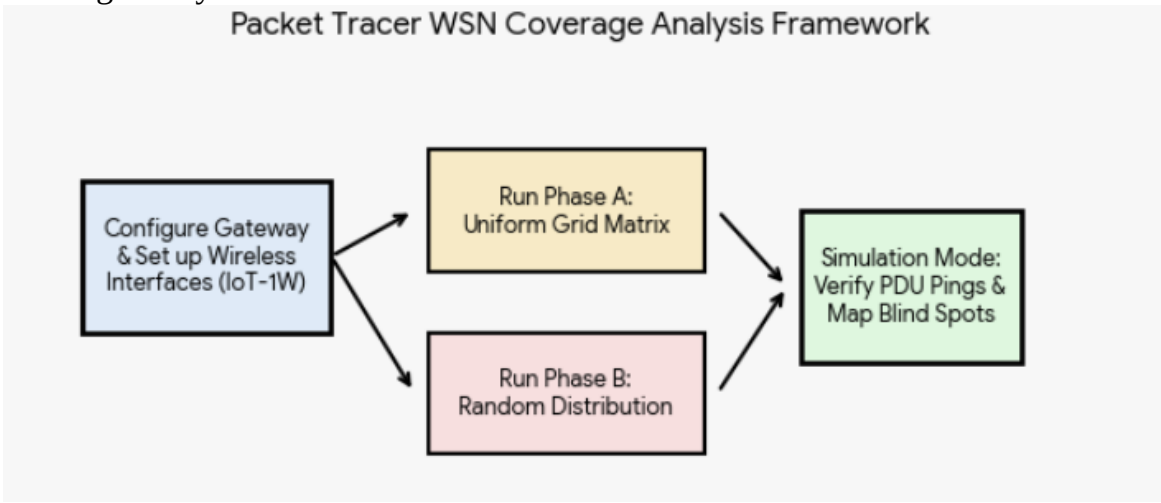
line with the gateway or a nearby neighbor node. Ping envelopes travel cleanly to the destination without drops, confirming full network coverage and connectivity.

- **Random Topology Analysis Observations:** The random deployment reveals typical ad-hoc network challenges. The clustered nodes close to the gateway experience high channel contention, while distant edge nodes fail to establish links entirely. Packet simulation runs show that pings from these isolated nodes drop immediately at the source interface, demonstrating localized **dead zones** caused by poor node distribution.
- **Conclusion:** This practical experiment successfully evaluates WSN deployment strategies using Cisco Packet Tracer. The simulation demonstrates that while random deployment is faster and cheaper, a structured grid deployment provides far better sensing coverage, more stable connectivity, and higher packet delivery success rates by eliminating network blind spots and communication gaps.

Architectural Layout & State Graphs

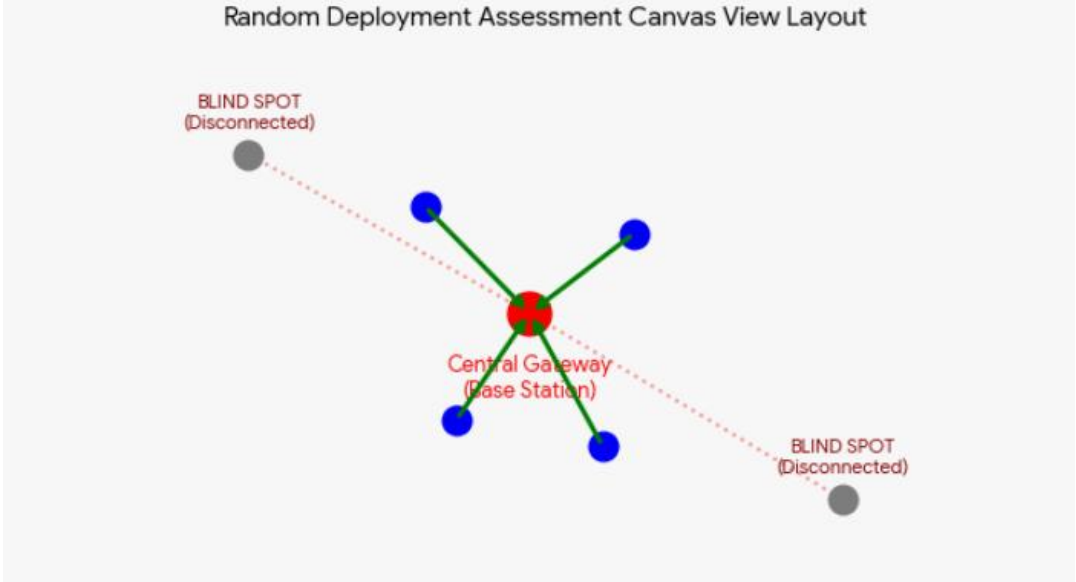
A. Simulation Execution Flowchart

This block diagram illustrates the processing steps required to run the deployment and coverage analysis:



B. Spatial Topology Link Assessment Model

This diagram maps out the visual link states observed during the random deployment scenario, showing the difference between connected clusters and isolated edge nodes:

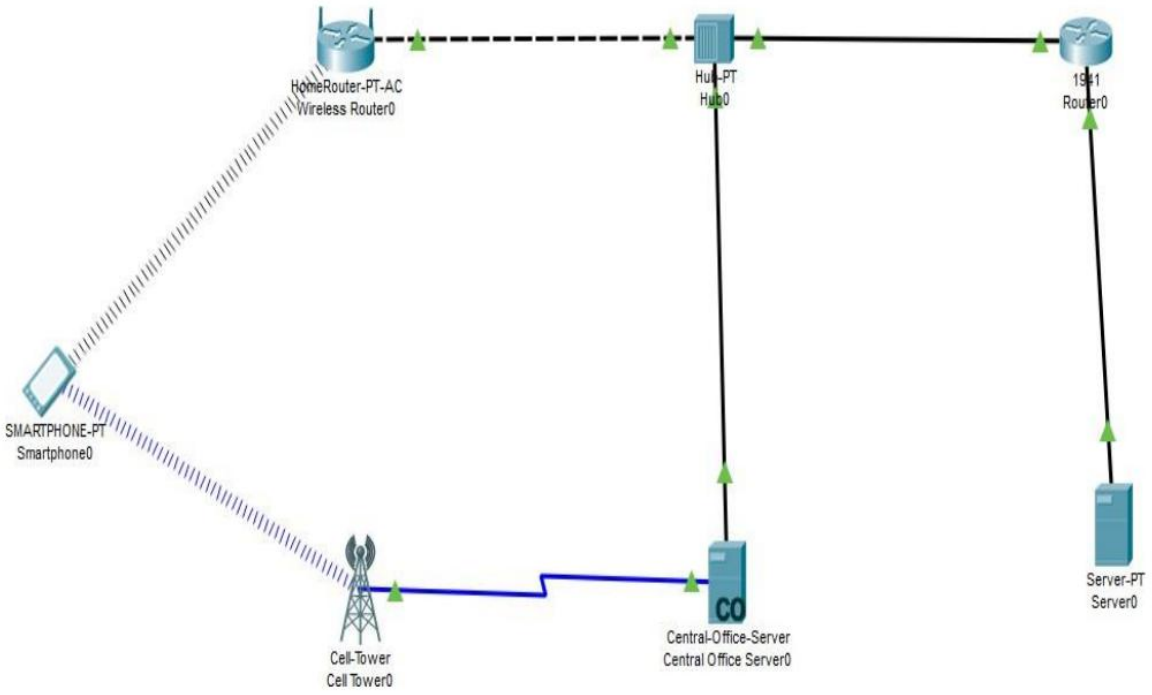


Practical No. 09

Aim: Cellular Network Simulation with Client-Server Communication: Simulate a mobile network consisting of a cell tower, central office server, web server, and web browser, and analyze packet flow during a data request-response cycle.

Theory:

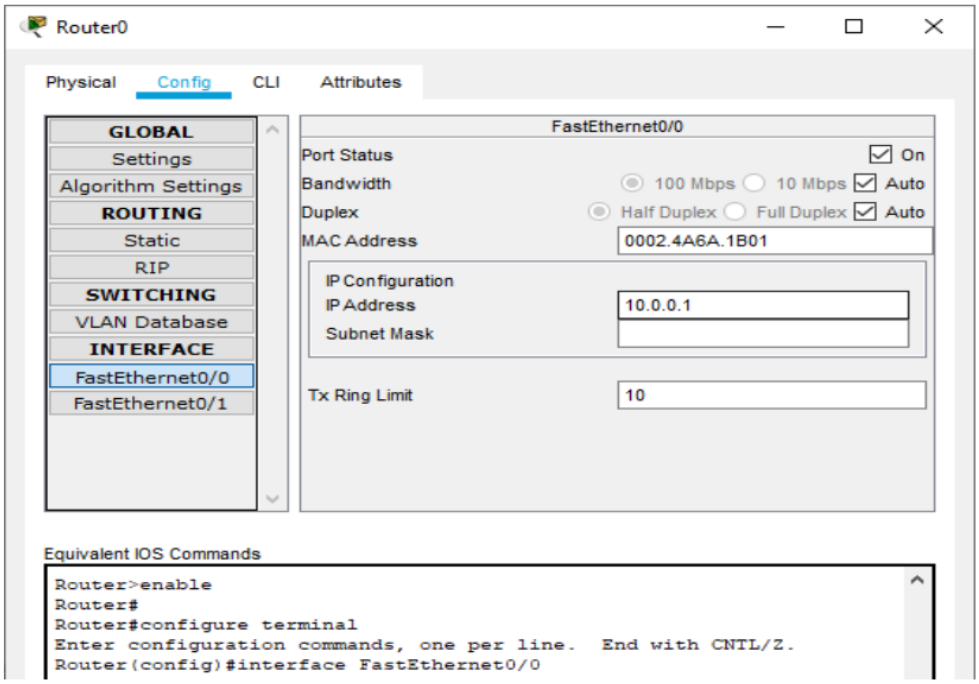
Consider the following topology



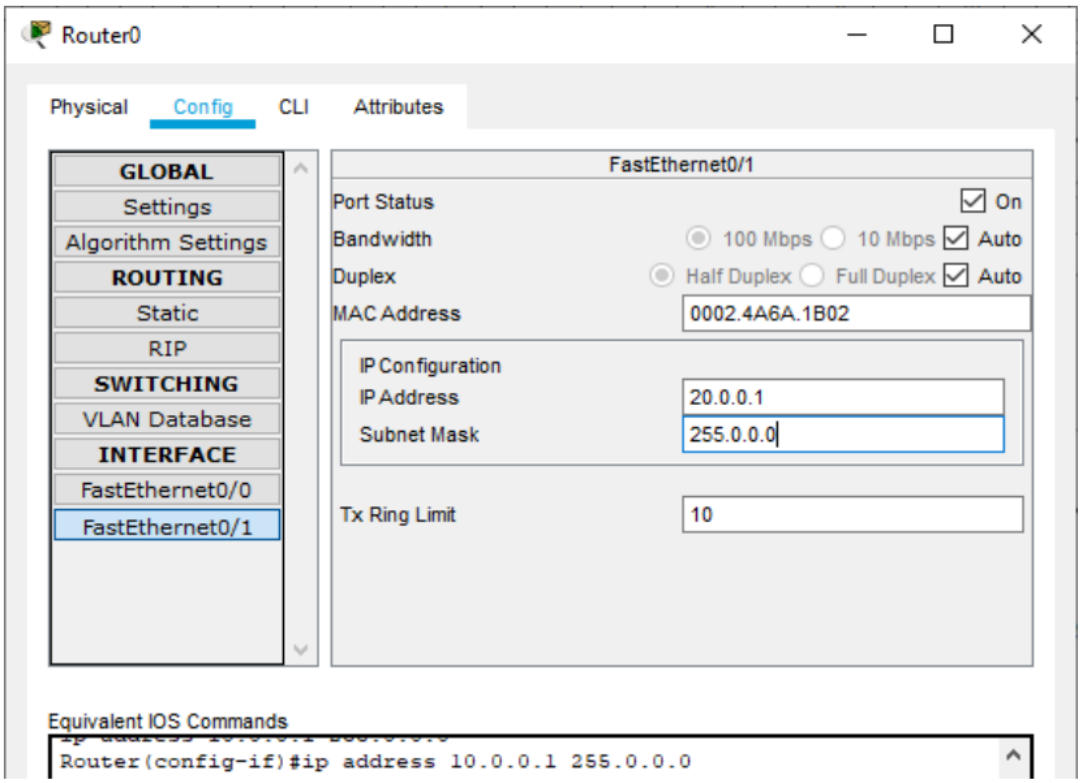
IP address configuration is done for the following devices

1) Router 0:

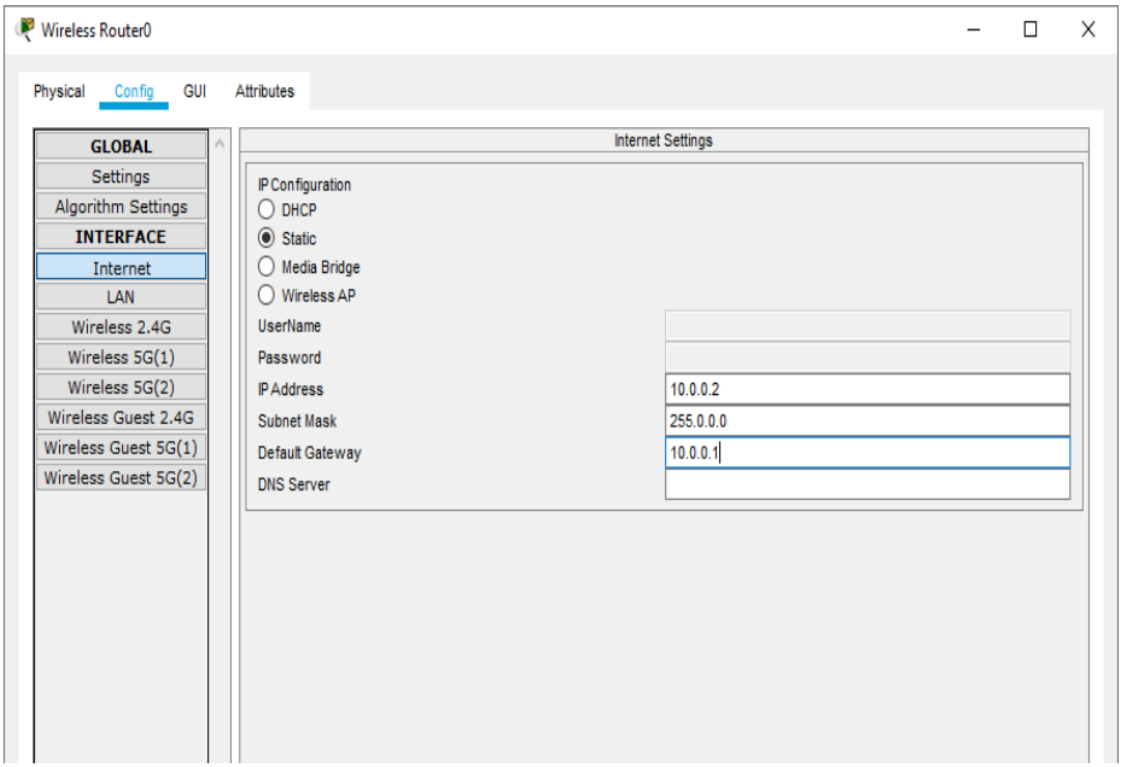
i) Interface: FastEthernet0/0:



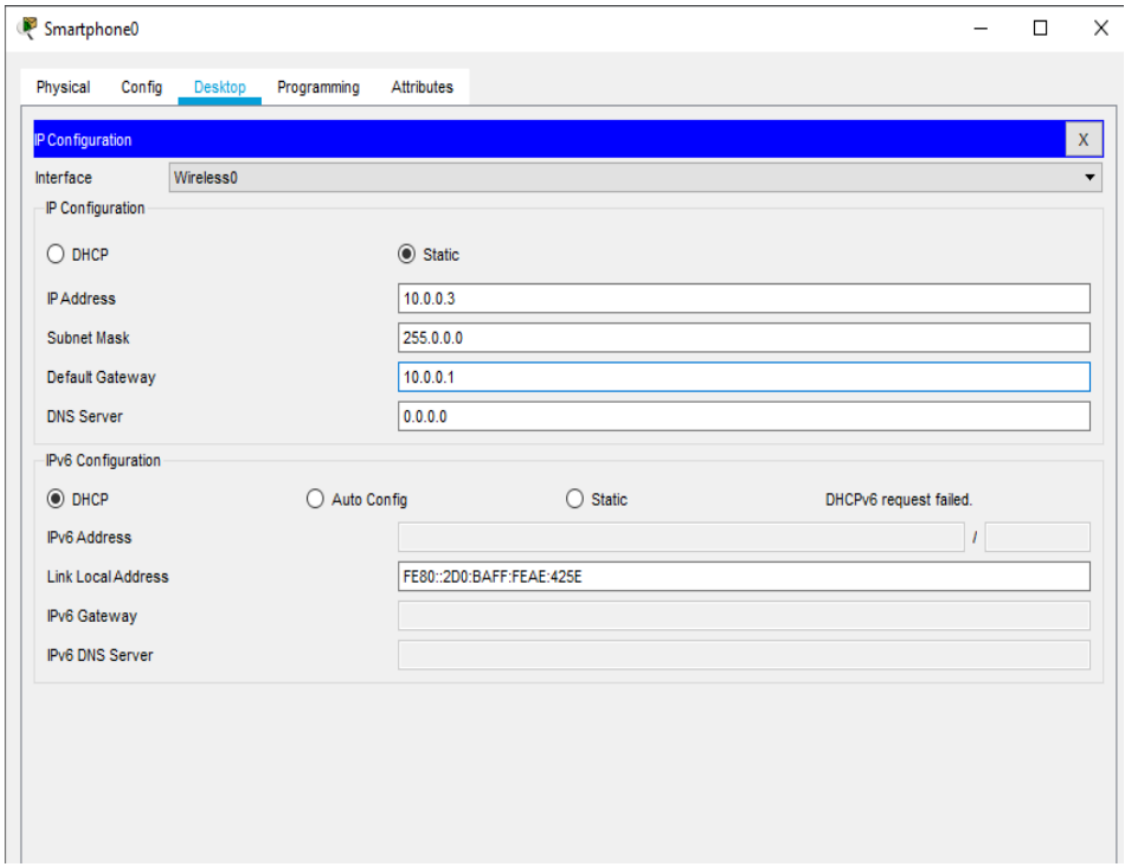
ii) Interface: FastEthernet0/1:



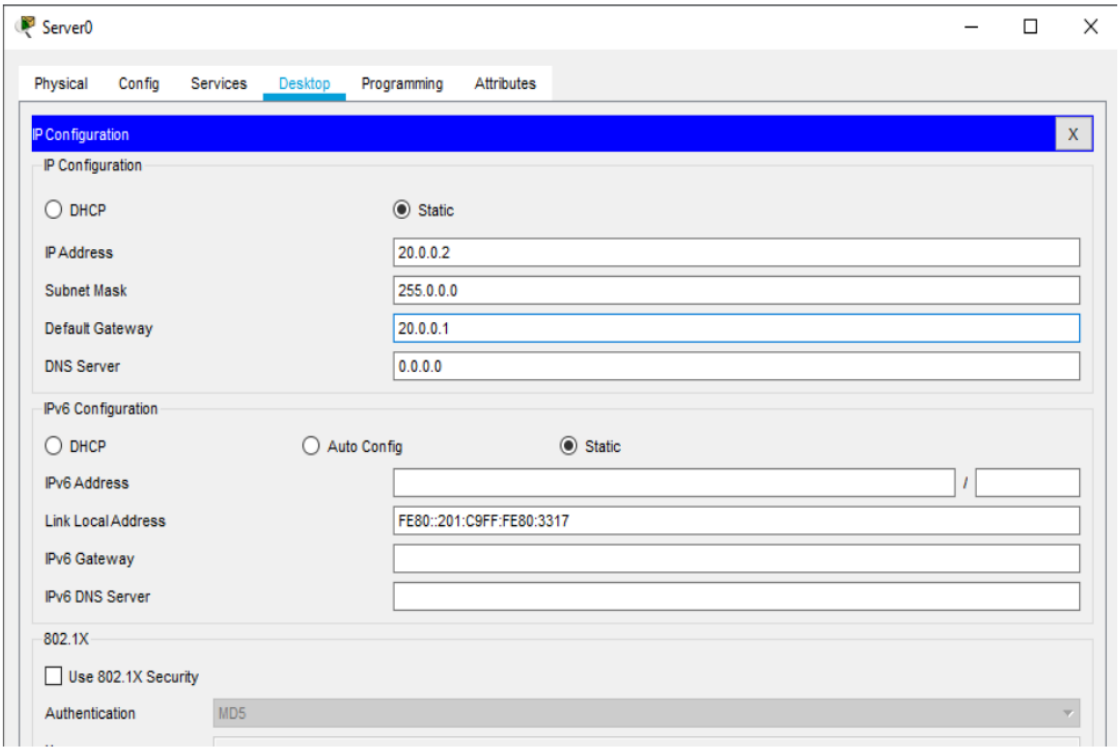
2) Wireless Router:



3) Smartphone:

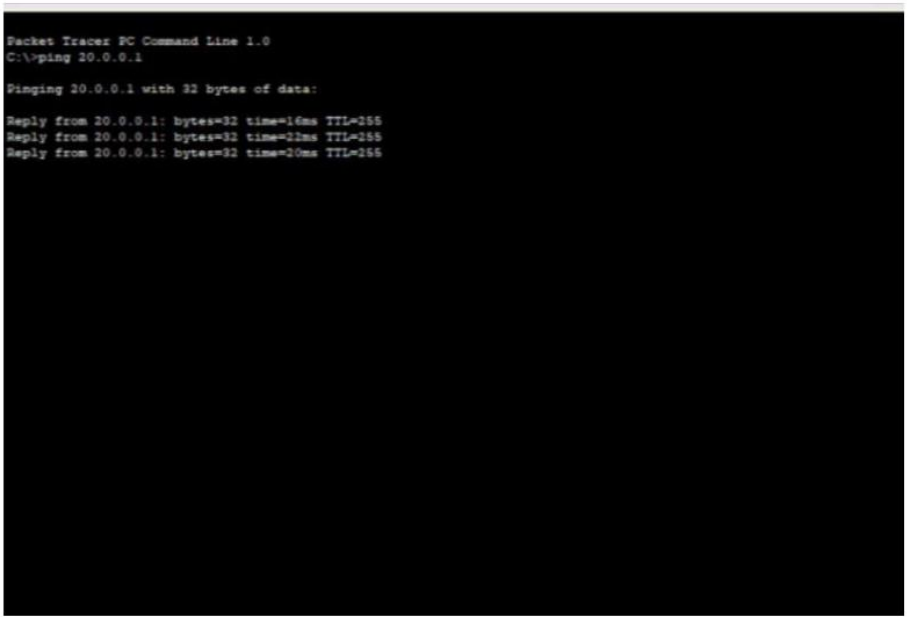


4) Server:

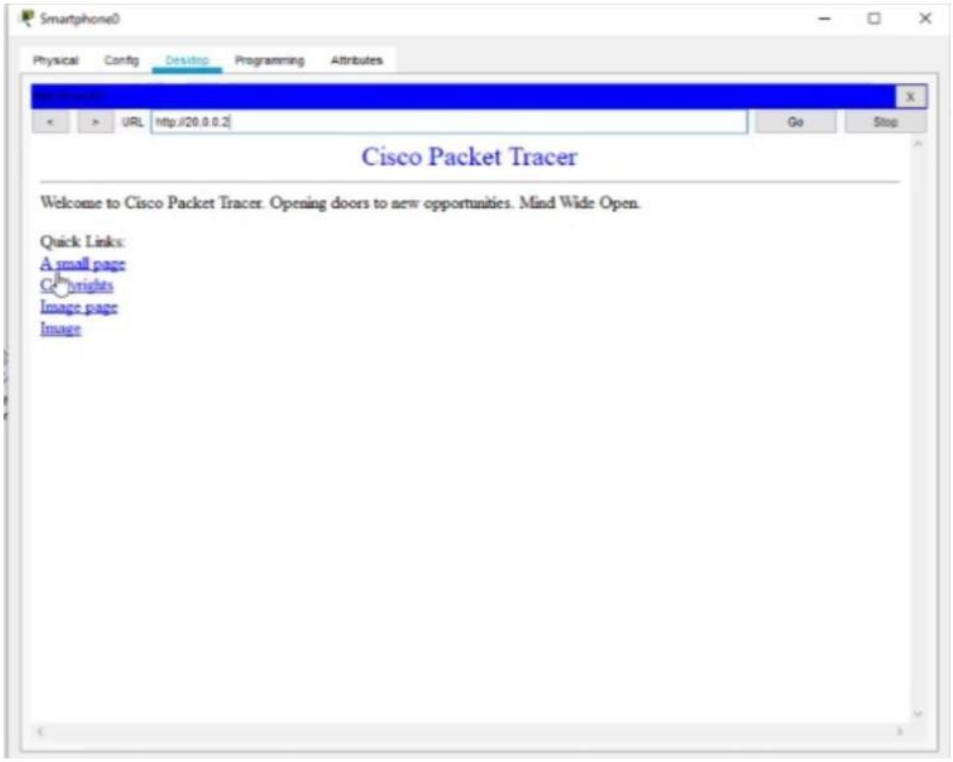


Next we verify the network connectivity as follows

1) Send a ping message from smartphone to server



2) Access the web service from the server through the Smartphone



Hence the mobile network was created and connectivity was also verified